Abstract

Analytical work in sports data often begins with a rapid download of the first available CSV file, followed by the production of recommendations. Yet such practices, while convenient, rarely capture the true complexity of the football industry or the structural dynamics that govern it. This notebook instead adopts a systematic approach: before proposing solutions, it examines historical market behavior to identify both structural weaknesses and enduring strengths within the global transfer system.

The methodology relies on rigorous data engineering. Multiple sources (Transfermarkt, Wyscout, Capology, Deloitte, among others) were consolidated, and extensive data cleaning was performed to address the high proportion of missing values (over 30% in key variables such as transfer fees and market valuations). A combined dataset was constructed to mitigate bias while preserving statistical representativeness. This enabled a reliable exploration of long-term transfer dynamics.

Empirical results confirm a strong correlation between market values and effective transfer fees (Pearson coefficient ≈ 0.93, $p < 0.001$), validating the use of market value as a predictive metric. At the same time, longitudinal analysis highlights rapid market expansion, with compound annual growth rates above 15% in both transfer activity and aggregated transaction volume during the past decade. Despite this expansion, profitability remains elusive: 98.3% of clubs report negative balances once wage expenditures are accounted for, indicating systemic inefficiency across the industry.

These findings provide not only a descriptive account of market evolution but also a foundation for designing prescriptive models. By detecting recurrent mispricings, identifying systematic inefficiencies, and quantifying structural imbalances, the analysis establishes the conditions for solutions that could enhance decision-making and address the sustainability challenges of professional football.

0. Data used:

- Transfermarkt: https://www.kaggle.com/datasets/davidcariboo/player-scores
- Wyscout Data: https://wyscout.hudl.com
- Other Sources (Wage Data):
  - https://fbref.com/en/comps/Big5/wages/Big-5-European-Leagues-Wages
  - https://www.transfermarkt.us/statistik/transfersalden
  - https://www.capology.com/uk/premier-league/salaries/
  - https://sportingintelligence832.substack.com/p/data-dive-the-finances-of-european
  - https://www.deloitte.com/fi/fi/about/press-room/annual-review-of-football-finance-2024.html
  - https://www.theguardian.com/football/2025/mar/06/premier-league-revenues-almost-double-those-in-la-liga-and-bundesliga

1. Data Loading

In [1]:

```python
# Import modules
import re
import ast
import numpy as np
import pandas as pd

from scipy.stats import pearsonr
from sklearn.cluster import KMeans
from scipy.spatial import ConvexHull
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler


import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

from pathlib import Path
from typing import Optional
```

In [2]:

```python
# Import modules
import plotly, plotly.io as pio

#from kaleido.scopes.plotly import PlotlyScope
#pio.kaleido.scope = PlotlyScope()

pio.renderers.default = "notebook"
```

In [3]:

```python
# Define data path
data_path = "../data"

# Load data
try:
    # Transfermarkt
    df_clubs = pd.read_csv(f"{data_path}/tm_clubs.csv")
    df_transfers = pd.read_csv(f"{data_path}/tm_transfers.csv")
    df_club_games = pd.read_csv(f"{data_path}/tm_club_games.csv")
    df_competitions = pd.read_csv(f"{data_path}/tm_competitions.csv")

    # Wyscout
    df_league_coverage = pd.read_csv(f"{data_path}/wyscout_league_coverage.csv")
    df_players = pd.read_csv(f"{data_path}/wyscout_players.csv")

    print("Files loaded successfully")
except FileNotFoundError:
    print(f"Error: One or more files at {data_path} were not found.")
    exit()
```

```
Files loaded successfully
```

In [4]:

```python
# Create output directory
image_path = Path("../outputs/plots")
image_path.mkdir(parents=True, exist_ok=True)
```

2. Data Cleaning

2.1. Data transformation

It is important to convert dates into a format that can be used to work with them.

In [5]:

```python
# Modification of transfer_season column to preserve order
def expand_season(season_str):
    start_year = int(season_str[:2])
    end_year = int(season_str[3:])

    if start_year < 70:  # Consider that are 20xx
        start_full = 2000 + start_year
        end_full = 2000 + end_year
    else:  # Consider that are 19xx
        start_full = 1900 + start_year
        end_full = 1900 + end_year

    return f"{start_full}/{str(end_full)[-2:]}"

df_transfers['transfer_season'] = df_transfers['transfer_season'].apply(expand_season)
```

Data from 2025 onwards must be deleted, as the data is incomplete as of August 2025. This will prevent bias in the results.

In [6]:

```python
# Convert transfer_date into datetime type
df_transfers['transfer_date'] = pd.to_datetime(df_transfers['transfer_date'], errors='coerce')

# Remove data from 2025 onward
df_transfers = df_transfers[
    df_transfers['transfer_date'] <= pd.Timestamp('2024-12-31')
].copy()

# Still remain noisy data from 2025 onward
df_transfers[
    df_transfers['transfer_season'].str.split('/').str[0].astype(int) >= 2025
]
```

Out[6]:

	player_id	transfer_date	transfer_season	from_club_id	to_club_id	from_club_name	to_club_name	transfer_fee	market_value_in_eur	player_name
5950	1027067	2024-07-01	2025/26	67278	67279	St. Johnst. U18	St. Johnst. B	NaN	NaN	Bayley Klimionek
6754	551752	2024-06-30	2025/26	8970	416	Frosinone	Torino	0.0	2000000.0	Demba Seck

In [7]:

```python
# Complete the cleaning process
df_transfers = df_transfers[
    df_transfers['transfer_season'].str.split('/').str[0].astype(int) <= 2024
].copy()

df_transfers[
    df_transfers['transfer_season'].str.split('/').str[0].astype(int) >= 2025
]
```

Out[7]:

player_id	transfer_date	transfer_season	from_club_id	to_club_id	from_club_name	to_club_name	transfer_fee	market_value_in_eur	player_name

2.2. Cleaning Missing Values

The treatment of missing values is essential in any data analysis. Therefore, we will now check which variables are missing in order to understand the nature of this dataset and perform a thorough cleanup.

In [8]:

```python
# Check missing values in the dataset
missing_values = df_transfers.isnull().sum()

# Display the missing values for each column
missing_values
```

Out[8]:

```
player_id                  0
transfer_date              0
transfer_season            0
from_club_id               0
to_club_id                 0
from_club_name             0
to_club_name               0
transfer_fee           27512
market_value_in_eur    30211
player_name                0
dtype: int64
```

In [9]:

```python
# Check how many rows have missing values in both columns, transfer_fee and market_value_in_eur
missing_both_values = df_transfers[
    df_transfers['transfer_fee'].isnull() & df_transfers['market_value_in_eur'].isnull()
]

# Show the result
len(missing_both_values)
```

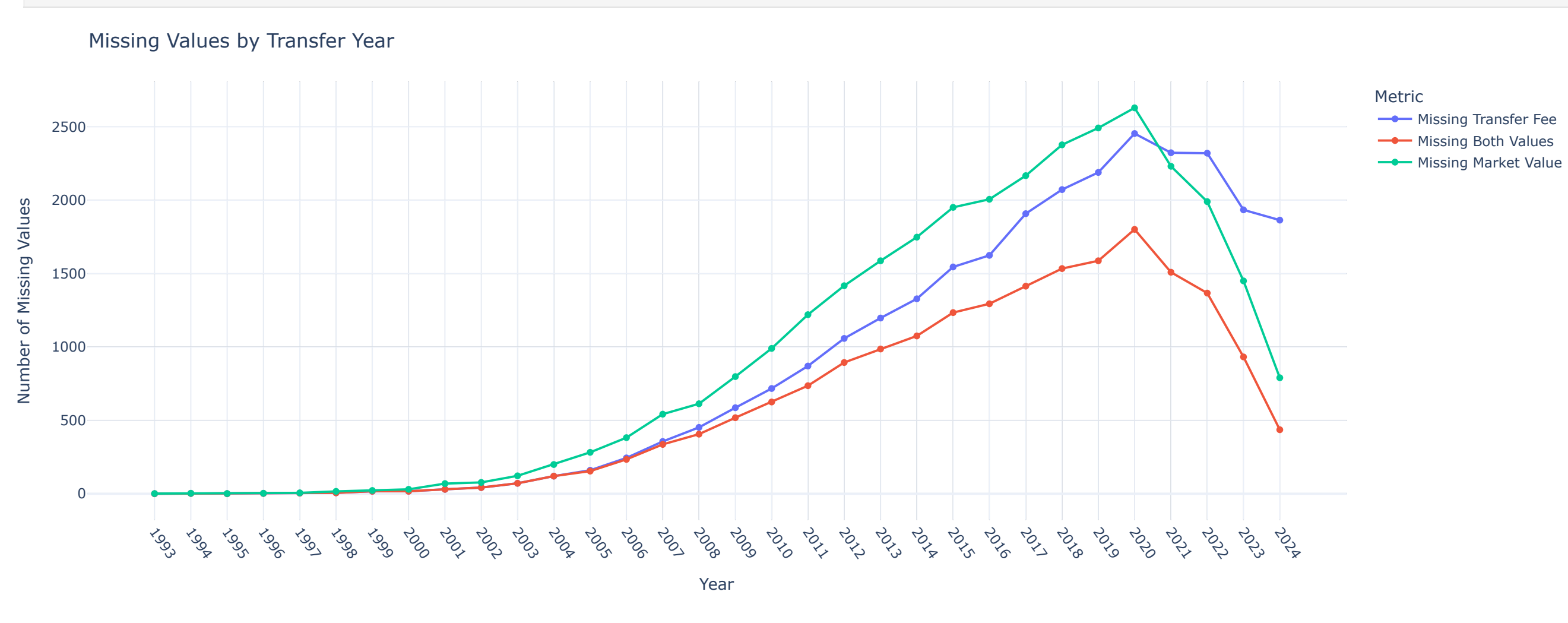
Out[9]:

```
19383
```

2.3. Missing Values per Year

In [10]:

```python
# Check missing values by year
missing_by_year = (
    df_transfers
    .groupby(df_transfers['transfer_date'].dt.year)[['transfer_fee', 'market_value_in_eur']]
    .apply(lambda group: pd.Series({
        'missing_transfer_fee': group['transfer_fee'].isnull().sum(),
        'missing_market_value': group['market_value_in_eur'].isnull().sum(),
        'missing_both_values': (
            group['transfer_fee'].isnull() &
            group['market_value_in_eur'].isnull()
        ).sum()
    }))
    .reset_index()
    .rename(columns={'transfer_date': 'year'})
)

# Create column of date for the continuous X axis
missing_by_year['year_date'] = pd.to_datetime(missing_by_year['year'].astype(str) + '-01-01')

# Convert to long format for plotly
df_long = missing_by_year.melt(
    id_vars=['year', 'year_date'],
    value_vars=['missing_transfer_fee', 'missing_market_value', 'missing_both_values'],
    var_name='metric',
    value_name='missing_count'
)

# Rename metrics for the legend
name_map = {
    'missing_transfer_fee': 'Missing Transfer Fee',
    'missing_market_value': 'Missing Market Value',
    'missing_both_values': 'Missing Both Values'
}
df_long['metric'] = df_long['metric'].map(name_map)

# Create figure
fig = px.line(
    df_long.sort_values('year_date'),
    x='year_date',
    y='missing_count',
    color='metric',
    markers=True,
    hover_data={'year': True, 'year_date': False}
)

# Layout
ordered = missing_by_year.sort_values('year_date')
fig.update_layout(
    title='Missing Values by Transfer Year',
    xaxis_title='Year',
    yaxis_title='Number of Missing Values',
    legend_title='Metric',
    template='plotly_white',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=ordered['year_date'],
        ticktext=ordered['year'].astype(str),
        tickangle=55
    )
)

fig.show()

fig.write_image(
    image_path / "missing_values_by_year.png",
    width=1280,
    height=720,
    scale=2
)
```



3. EDA (Exploratory Data Analysis)

3.1. Creation of Clean Datasets

Depending on the analysis to be performed on each occasion, it is important to have the appropriate dataset. That is why different datasets will be created, which will be used at the most appropriate time for each one.

3.1.1. Cleaned-Both Dataset: Created by removing any row with missing values in *transfer_fee* or *market_value_in_eur*

On this scenario, it will be necessary to ensure that the dataset is as accurate as possible, while taking into account the handicap of having fewer records available.

In [11]:

```python
# Create a new dataframe with the cleaned data
df_transfers_cleaned_both = df_transfers.dropna(
    subset=['transfer_fee', 'market_value_in_eur'],
    how='any'
).copy()

# Show the dataset information that have been recently cleaned
print(f"Dataset Shape: {df_transfers_cleaned_both.shape}")
# print(f"\nColumn Data Types:\n{df_transfers_cleaned_both.dtypes}")
print(f"\nMissing Values:\n{df_transfers_cleaned_both.isnull().sum()}")
```

```
Dataset Shape: (39378, 10)

Missing Values:
player_id              0
transfer_date          0
transfer_season        0
from_club_id           0
to_club_id             0
from_club_name         0
to_club_name           0
transfer_fee           0
market_value_in_eur    0
player_name            0
dtype: int64
```

3.1.2. No-Missing-Fee Dataset: Removing every row with any missing value in *transfer_fee*

In this scenario, it will retain all transfers that contain information about the *transfer_fee* for the transaction.

In [12]:

```python
# Remove any row where transfer_fee is missing
df_transfers_no_missing_fee = df_transfers.dropna(subset=['transfer_fee']).copy()

# Show the dataset information that have been recently cleaned
print(f"Dataset Shape: {df_transfers_no_missing_fee.shape}")
# print(f"\nColumn Data Types:\n{df_transfers_no_missing_fee.dtypes}")
print(f"\nMissing Values:\n{df_transfers_no_missing_fee.isnull().sum()}")
```

```
Dataset Shape: (50206, 10)

Missing Values:
player_id                  0
transfer_date              0
transfer_season            0
from_club_id               0
to_club_id                 0
from_club_name             0
to_club_name               0
transfer_fee               0
market_value_in_eur    10828
player_name                0
dtype: int64
```

3.1.3. No-Missing-MV (Market Value) Dataset: Removing every row with any missing value in *market_value_in_eur*

In this scenario, it will retain all transfers that contain information about the *market_value_in_eur* of the player involved in the transaction.

In [13]:

```python
# Remove any row where transfer_fee is missing
df_transfers_no_missing_mv = df_transfers.dropna(subset=['market_value_in_eur']).copy()

# Show the dataset information that have been recently cleaned
print(f"Dataset Shape: {df_transfers_no_missing_mv.shape}")
# print(f"\nColumn Data Types:\n{df_transfers_no_missing_mv.dtypes}")
print(f"\nMissing Values:\n{df_transfers_no_missing_mv.isnull().sum()}")
```

```
Dataset Shape: (47507, 10)

Missing Values:
player_id                 0
transfer_date             0
transfer_season           0
from_club_id              0
to_club_id                0
from_club_name            0
to_club_name              0
transfer_fee           8129
market_value_in_eur       0
player_name               0
dtype: int64
```

3.2. Comparison of Datasets

The purpose of this section is to understand how transfers are distributed by transfer fee, which will help us to better understand the different datasets we have generated. To do this, we must ask ourselves which dataset is most appropriate for this analysis.

3.2.1. Cleaned-Both vs No-Misssing-Fee

It will begin by comparing the following datasets:

- **Cleaned-Both**: Dataset created by removing all rows missing the *transfer_fee* or *market_value_in_eur*, resulting in a completely clean dataset.
- **No-Missing-Fee**: Dataset created by removing all rows with missing *transfer_fee*, regardless of whether the *market_value_in_eur* is missing, resulting in a dataset with no missing values in the *transfer_fee* column.

The comparison will focus on the difference in the distribution of transfer fees between them.

In [14]:

```python
# Count total number of transfers
total_both = df_transfers_cleaned_both.shape[0]
total_no_missing_fee = df_transfers_no_missing_fee.shape[0]

# Filter and count free transfers (transfer_fee == 0)
free_both = (df_transfers_cleaned_both['transfer_fee'] == 0).sum()
free_no_missing_fee = (df_transfers_no_missing_fee['transfer_fee'] == 0).sum()

# Filter and count low-cost transfers but not free (0 < transfer_fee <= 100000)
low_cost_both = ((df_transfers_cleaned_both['transfer_fee'] > 0) & (df_transfers_cleaned_both['transfer_fee'] <= 100000)).sum()
low_cost_no_missing_fee = ((df_transfers_no_missing_fee['transfer_fee'] > 0) & (df_transfers_no_missing_fee['transfer_fee'] <= 100000)).sum()

# Show summary
print(f"Total number of transfers:")
print(f"\tCleaned Both   : {total_both:,}")
print(f"\tNo Missing Fee : {total_no_missing_fee:,}")
print(f"\tDifference     : {total_no_missing_fee - total_both:,}")

print("\n")
print("Free transfers (transfer_fee = 0):")
print(f"\tCleaned Both   : {free_both:,}")
print(f"\tNo Missing Fee : {free_no_missing_fee:,}")
print(f"\tDifference     : {free_no_missing_fee - free_both:,}")

print("\n")
print("Low-cost transfers (0 < transfer_fee <= 100000):")
print(f"\tCleaned Both   : {low_cost_both:,}")
print(f"\tNo Missing Fee : {low_cost_no_missing_fee:,}")
print(f"\tDifference     : {low_cost_no_missing_fee - low_cost_both:,}")

print("\n")
print(f"Percentage of free transfers of the additional entries in the No Missing Fee dataset: {(free_no_missing_fee - free_both) / (total_no_missing_fee - total_both) * 100:.2f}
print(f"Percentage of low-cost transfers of the additional entries in the No Missing Fee dataset: {(low_cost_no_missing_fee - low_cost_both) / (total_no_missing_fee - total_both
```

```
Total number of transfers:
        Cleaned Both   : 39,378
        No Missing Fee : 50,206
        Difference     : 10,828


Free transfers (transfer_fee = 0):
        Cleaned Both   : 30,792
        No Missing Fee : 40,991
        Difference     : 10,199


Low-cost transfers (0 < transfer_fee <= 100000):
        Cleaned Both   : 422
        No Missing Fee : 615
        Difference     : 193


Percentage of free transfers of the additional entries in the No Missing Fee dataset: 94.19%
Percentage of low-cost transfers of the additional entries in the No Missing Fee dataset: 1.78%
```

The additional rows found in the No-Missing-Fee dataset and not found in Cleaned-Both are rows that have a *transfer_fee* but no *market_value_in_eur*. Of these additional rows, approximately 94.2% of transfers were free, and approximately 1.8% were low-cost transfers (less or equal than 100k €).

In other words, approximately 96% of the transfers added to the No-Missing-Fee dataset are free or low-cost. We can assume that these transfers lack *market_value_in_eur* because they involve unknown players, players from lower categories, or players with a very low market value.

That is why we conclude that the No-Missing-Fee dataset does not provide us with any additional information to the Cleaned-Both dataset, beyond entries for volume.

3.2.2. Cleaned-Both vs No-Misssing-MV (Market Value)

It will continue comparing the following datasets:

- **Cleaned-Both**: Dataset created by removing all rows missing the *transfer_fee* or *market_value_in_eur*, resulting in a completely clean dataset.
- **No-Missing-MV**: Dataset created by removing all rows missing the *market_value_in_eur*, regardless of whether the *transfer_fee* is missing, resulting in a dataset with no missing values in the *market_value_in_eur* column.

The comparison will focus on the difference in the distribution of market values between them.

In [15]:

```python
# Count total number of transfers
total_both = df_transfers_cleaned_both.shape[0]
total_no_missing_mv = df_transfers_no_missing_mv.shape[0]

# Filter and count transfers with market value lower than 100000 (market_value_in_eur <= 100000)
mv_100k_both = ((df_transfers_cleaned_both['market_value_in_eur'] <= 100000)).sum()
mv_100k_no_missing_mv = ((df_transfers_no_missing_mv['market_value_in_eur'] <= 100000)).sum()

retired_players = df_transfers_cleaned_both[df_transfers_cleaned_both['to_club_name'] == 'Retired'].shape[0]
retired_players_no_missing_mv = df_transfers_no_missing_mv[df_transfers_no_missing_mv['to_club_name'] == 'Retired'].shape[0]

# Show summary
print(f"Total number of transfers:")
print(f"\tCleaned Both   : {total_both:,}")
print(f"\tNo Missing MV  : {total_no_missing_mv:,}")
print(f"\tDifference     : {total_no_missing_mv - total_both:,}")

print("\n")
print("Transfers with market value lower than 100000 (market_value_in_eur <= 100000):")
print(f"\tCleaned Both   : {mv_100k_both:,}")
print(f"\tNo Missing MV  : {mv_100k_no_missing_mv:,}")
print(f"\tDifference     : {mv_100k_no_missing_mv - mv_100k_both:,}")

print("\n")
print("Transfers with market value that appears as a retired player:")
print(f"\tCleaned Both   : {retired_players:,}")
print(f"\tNo Missing MV  : {retired_players_no_missing_mv:,}")
print(f"\tDifference     : {retired_players_no_missing_mv - retired_players:,}")

print("\n")
print(f"Percentage of transfers with market value lower than 100000 of the additional entries in the No Missing MV dataset: {(mv_100k_no_missing_mv - mv_100k_both) / (total_no_m
print(f"Percentage of transfers with market value that retire of the additional entries in the No Missing MV dataset: {(retired_players_no_missing_mv - retired_players) / (total
```

```
Total number of transfers:
        Cleaned Both   : 39,378
        No Missing MV  : 47,507
        Difference     : 8,129


Transfers with market value lower than 100000 (market_value_in_eur <= 100000):
        Cleaned Both   : 4,513
        No Missing MV  : 6,864
        Difference     : 2,351


Transfers with market value that appears as a retired player:
        Cleaned Both   : 0
        No Missing MV  : 58
        Difference     : 58


Percentage of transfers with market value lower than 100000 of the additional entries in the No Missing MV dataset: 28.92%
Percentage of transfers with market value that retire of the additional entries in the No Missing MV dataset: 0.71%
```

It is concluded that the additional rows found in the No-Missing-MV dataset and not found in Cleaned-Both are rows that have *market_value_in_eur* but not *transfer_fee*. Of these additional rows, approximately 29% of the transfers have been of players with low market value.

It could be assumed that those entries that have *market_value_in_eur* but no *transfer_fee* are due to players retiring, but these players account for around 0.7% of the additional entries in No-Missing-MV, so this hypothesis is ruled out.

In this case, the result is not significant enough to suggest that the reason for entries missing the *transfer_fee* but not the *market_value_in_eur* is because they have an extremely low market value or because they are retiring players. That is why it is assumed that their *transfer_fee* has not been added due to a lack of information.

Therefore, it can be assumed that the *transfer_fee* for these transactions has been their *market_value_in_eur*, so that generality is not lost and we can expand the sample.

3.3. Combined Dataset: Creating the Final Dataset

After this cleaning process, the following conclusions have been reached:

- If a row lacks both *market_value_in_eur* and *transfer_fee*, we cannot extract any information other than the transfer itself for volumetry.
- If a row lacks *transfer_fee* but not *market_value_in_eur*, it is reasonable to consider that the transfer was made for the *market_value_in_eur*.
- If a row lacks *market_value_in_eur* but not *transfer_fee*, it is not correct to consider that the *market_value_in_eur* is similar to the *transfer_fee*.

Therefore, the dataset that will be called Combined Dataset will be created following these three rules.

In [16]:

```python
# Create a new dataframe with the cleaned data
df_transfers_combined = df_transfers.copy()

# 1) Eliminate rows where both values are missing (fee and mv)
df_transfers_combined = df_transfers_combined.dropna(subset=['transfer_fee', 'market_value_in_eur'], how='all').copy()

# 2) If transfer_fee is missing but market_value is present -> use mv as fee
df_transfers_combined['transfer_fee'] = df_transfers_combined['transfer_fee'].fillna(df_transfers_combined['market_value_in_eur'])

# 3) If market_value is missing but fee is present -> do not fill anything (remains NaN)

# Show the dataset information that have been recently cleaned
print(f"Dataset Shape: {df_transfers_combined.shape}")
# print(f"\nColumn Data Types:\n{df_transfers_combined.dtypes}")
print(f"\nMissing Values:\n{df_transfers_combined.isnull().sum()}")
```

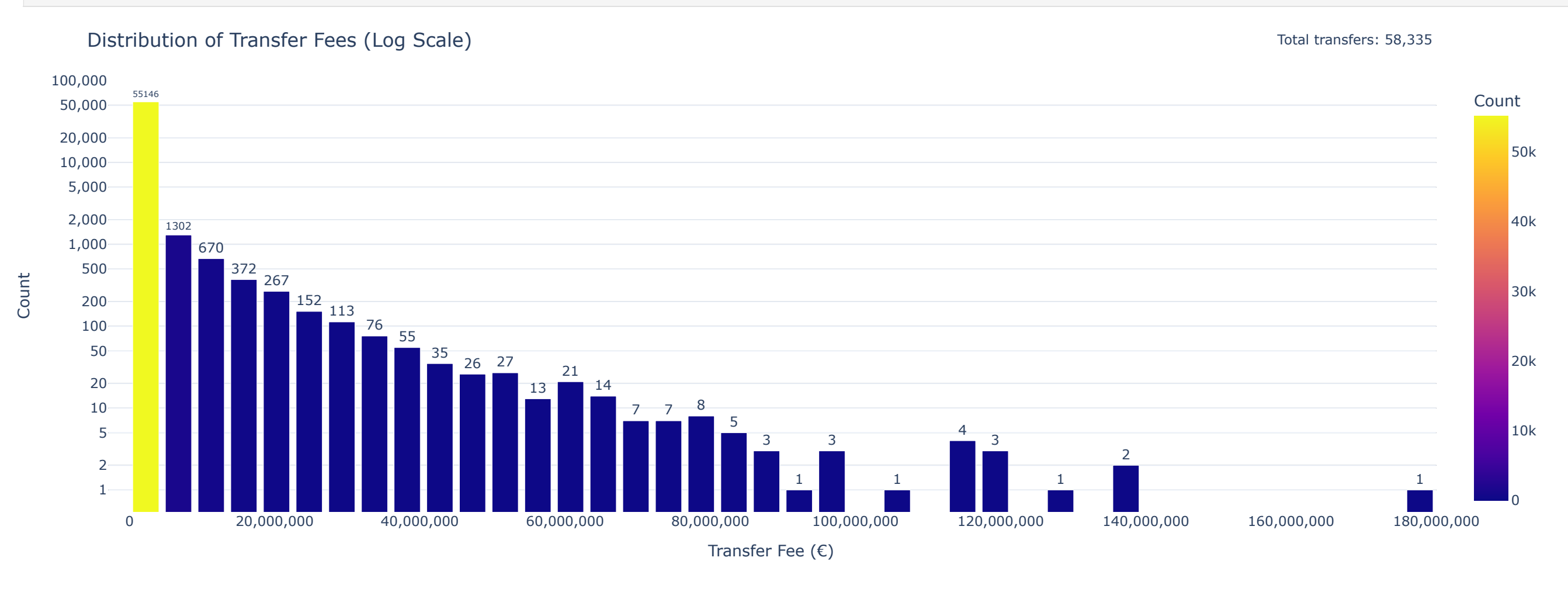
```
Dataset Shape: (58335, 10)

Missing Values:
player_id                  0
transfer_date              0
transfer_season            0
from_club_id               0
to_club_id                 0
from_club_name             0
to_club_name               0
transfer_fee               0
market_value_in_eur    10828
player_name                0
dtype: int64
```

3.3. Numeric Distribution of Transfer Fees

To find out how transactions are distributed according to their *transfer_fee*, the Combined Dataset will be used. This information is important for understanding the nature of the transfer market.

In [17]:

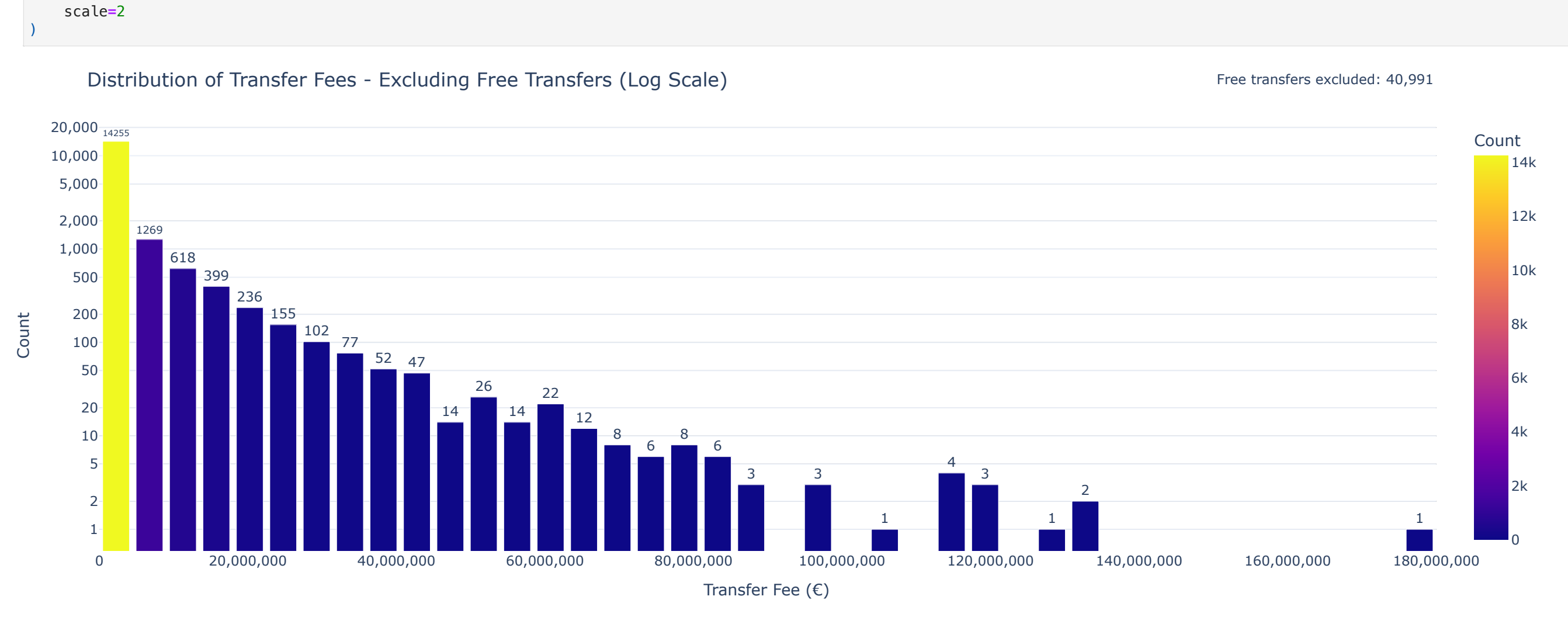
```python
# Prepare data (only valid transfer_fee)
series = pd.to_numeric(df_transfers_combined['transfer_fee'], errors='coerce').dropna()
if series.empty:
    raise ValueError("There are no valid values in 'transfer_fee'.")

# Statistics
mean_val = series.mean()
median_val = series.median()

# Manual binning to color by intensity
nbins = 40
counts, bin_edges = np.histogram(series, bins=nbins)
bin_left = bin_edges[:-1]
bin_right = bin_edges[1:]
bin_center = (bin_left + bin_right) / 2

df_bins = pd.DataFrame({
    'bin_left': bin_left,
    'bin_right': bin_right,
    'bin_center': bin_center,
    'count': counts
})

# Interactive plot
fig = px.bar(
    df_bins,
    x='bin_center',
    y='count',
    color='count',
    text='count',
    title='Distribution of Transfer Fees (Log Scale)',
    labels={'bin_center': 'Transfer Fee (€)', 'count': 'Count'},
    hover_data={'bin_left': ':.0f', 'bin_right': ':.0f', 'bin_center': ':.0f', 'count': ':.0f'}
)

fig.update_traces(textposition='outside', cliponaxis=False)

# Add peak annotation
idx_max = df_bins['count'].idxmax()
fig.add_annotation(
    x=df_bins.loc[idx_max, 'bin_center'],
    y=df_bins.loc[idx_max, 'count'],
    text=f"Peak<br>{int(df_bins.loc[idx_max, 'count']):,}",
    showarrow=True,
    arrowhead=2,
    yshift=10
)

# Style
fig.update_layout(
    template='plotly_white',
    xaxis_title='Transfer Fee (€)',
    yaxis_title='Count',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    legend=dict(
        orientation='v',
        yanchor='top', y=1,
        xanchor='left', x=0
    )
)

# Add note with number of total transfers
fig.add_annotation(
    x=1, y=1.12, xref='paper', yref='paper',
    text=f"Total transfers: {series.shape[0]:,}",
    showarrow=False, align='right'
)

# X axis with more compact format
fig.update_xaxes(tickformat=',.0f')
fig.update_yaxes(tickformat=',.0f')
fig.update_yaxes(type='log')

fig.show()

fig.write_image(
    image_path / "distribution_of_transfer_fees.png",
    width=1280,
    height=720,
    scale=2
)
```



Since free transfers account for the vast majority of transfers, we filter out those that are not free in order to better understand the fee-based market.

In [18]:

```python
# Filter: only valid transfer_fee and > 0 (exclude free)
series_all = pd.to_numeric(df_transfers_combined['transfer_fee'], errors='coerce').dropna()
series = series_all[series_all > 0]

if series.empty:
    raise ValueError("There are no valid values in 'transfer_fee' > 0 (excluding free).")

n_free = (series_all == 0).sum()

# Statistics (excluding free)
mean_val = series.mean()
median_val = series.median()

# 3) Bineado manual para colorear por intensidad
nbins = 40
counts, bin_edges = np.histogram(series, bins=nbins)
bin_left = bin_edges[:-1]
bin_right = bin_edges[1:]
bin_center = (bin_left + bin_right) / 2

df_bins = pd.DataFrame({
    'bin_left': bin_left,
    'bin_right': bin_right,
    'bin_center': bin_center,
    'count': counts
})

# Interactive plot
fig = px.bar(
    df_bins,
    x='bin_center',
    y='count',
    color='count',
    text='count',
    title='Distribution of Transfer Fees - Excluding Free Transfers (Log Scale)',
    labels={'bin_center': 'Transfer Fee (€)', 'count': 'Count'},
    hover_data={'bin_left': ':.0f', 'bin_right': ':.0f', 'bin_center': ':.0f', 'count': ':.0f'}
)

fig.update_traces(textposition='outside', cliponaxis=False)

# Style
fig.update_layout(
    template='plotly_white',
    xaxis_title='Transfer Fee (€)',
    yaxis_title='Count',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    legend=dict(orientation='v', yanchor='top', y=1, xanchor='left', x=0)
)

fig.update_xaxes(tickformat=',.0f')
fig.update_yaxes(tickformat=',.0f')
fig.update_yaxes(type='log')

# Add note with number of free excluded
if n_free > 0:
    fig.add_annotation(
        x=1, y=1.12, xref='paper', yref='paper',
        text=f"Free transfers excluded: {n_free:,}",
        showarrow=False, align='right'
    )

fig.show()

fig.write_image(
    image_path / "distribution_of_transfer_fees_excluding_free.png",
    width=1280,
    height=720,
    scale=2
)
```

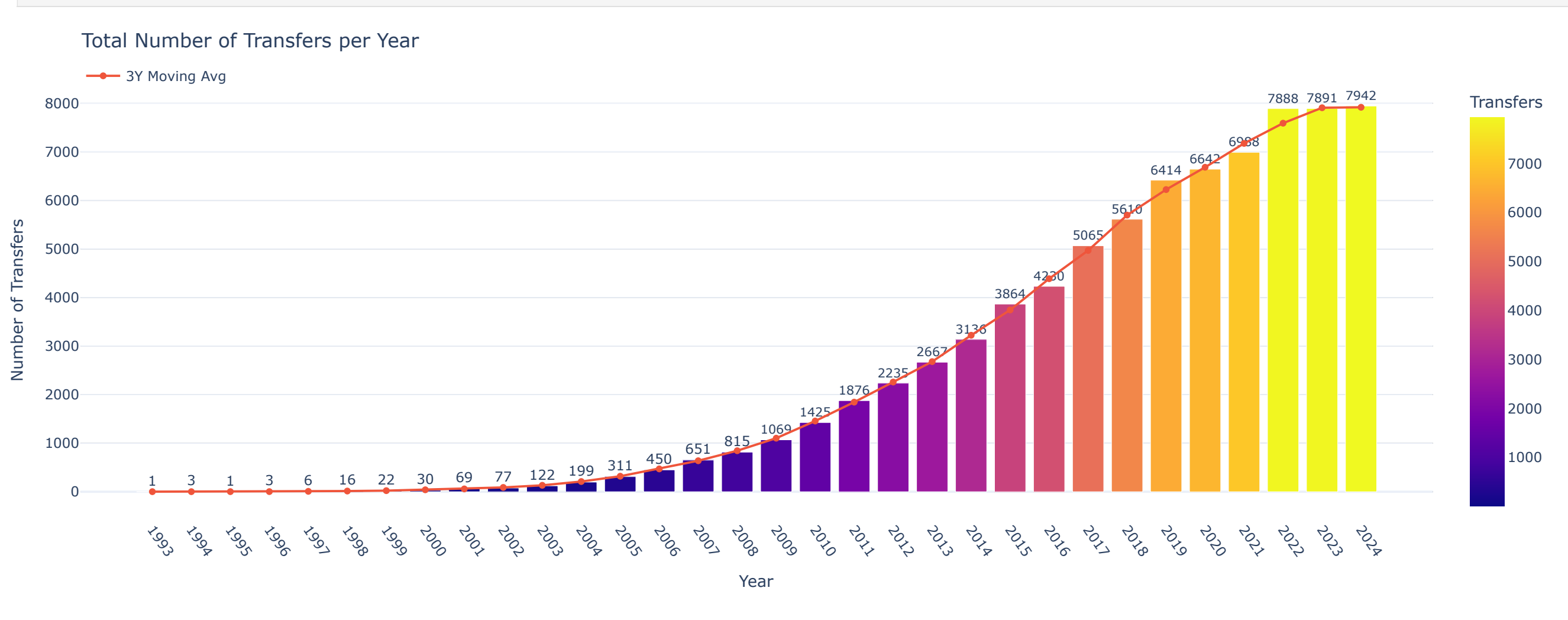


3.4. Number of Transfers per Year

The complete original dataset (uncleaned) is used to check the total number of free transfers per year.

In [19]:

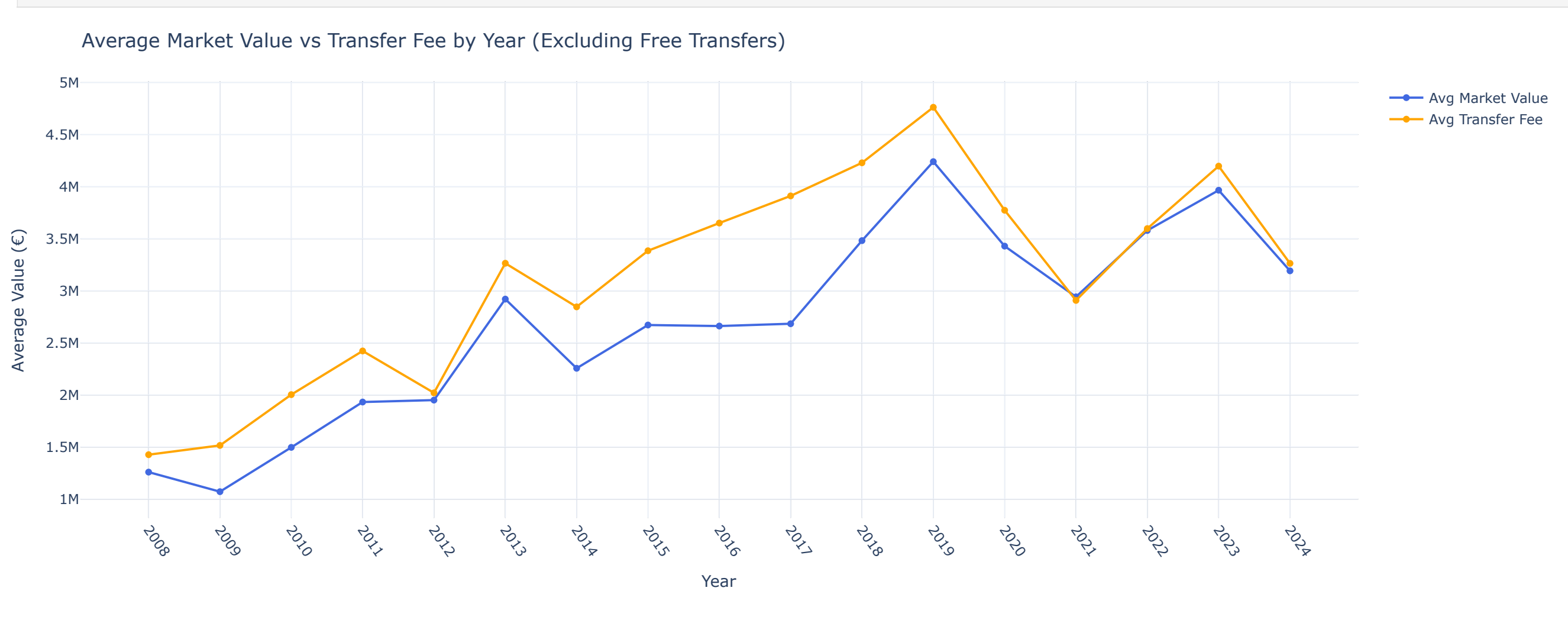
```python
# Count by year
counts = (
    df_transfers['transfer_date']
    .dt.year
    .value_counts()
    .rename_axis('year')
    .sort_index()
    .reset_index(name='n_transfers')
)

# 3-year rolling average
counts['roll3'] = counts['n_transfers'].rolling(3, center=True, min_periods=2).mean()

# Interactive plot
fig = px.bar(
    counts,
    x='year',
    y='n_transfers',
    color='n_transfers',
    text='n_transfers',
    title='Total Number of Transfers per Year',
    hover_data={'year': True}
)
fig.update_traces(textposition='outside', cliponaxis=False)

# Add trend line
fig.add_trace(go.Scatter(
    x=counts['year'],
    y=counts['roll3'],
    mode='lines+markers',
    name='3Y Moving Avg',
    hovertemplate='Year: %{x}<br>3Y Moving Avg: %{y:.0f}<extra></extra>'
))

# Style
fig.update_layout(
  legend=dict(
        orientation='v',
        yanchor='top',
        y=1.05,
        xanchor='left',
        x=0
    ),
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Number of Transfers',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=counts['year'],
        ticktext=counts['year'],
        tickangle=55,
        rangeslider=dict(visible=False),
        rangeselector=dict(visible=False)
    ),
    coloraxis_colorbar=dict(title='Transfers')
)

fig.show()

fig.write_image(
    image_path / "total_transfers_per_year.png",
    width=1280,
    height=720,
    scale=2
)
```



3.5. Correlation between Market Value and Transfer Fees

It is important to understand how market value correlates with the actual transfer price of players in order to determine whether market value is an effective metric for valuing players.

In [20]:
```
# Filter and group by year
df_yearly = (
    df_transfers_combined
      .loc[
          (df_transfers_combined['transfer_fee'] > 0)
      ]
      .dropna(subset=['transfer_fee', 'market_value_in_eur', 'transfer_date'])
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'] >= 2008]
      .groupby('year', as_index=False)
      .agg({
          'market_value_in_eur': 'mean',
          'transfer_fee': 'mean'
      })
)

# Create figure with two lines
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=df_yearly['year'],
    y=df_yearly['market_value_in_eur'],
    mode='lines+markers',
    name='Avg Market Value',
    line=dict(color='royalblue', width=2)
))

fig.add_trace(go.Scatter(
    x=df_yearly['year'],
    y=df_yearly['transfer_fee'],
    mode='lines+markers',
    name='Avg Transfer Fee',
    line=dict(color='orange', width=2)
))

# Style the graph
fig.update_layout(
    title='Average Market Value vs Transfer Fee by Year (Excluding Free Transfers)',
    xaxis_title='Year',
    yaxis_title='Average Value (€)',
    template='plotly_white',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='linear',
        dtick=1,
        tickangle=55
    )
)

fig.show()

fig.write_image(
    image_path / "market_value_vs_transfer_fee.png",
    width=1280,
    height=720,
    scale=2
)
```



It is also important to know the Pearson coefficient between Transfer Fee and Market Value to understand how they correlate mathematically.

In [21]:
```
# Extraer las dos series
x = df_yearly['market_value_in_eur']
y = df_yearly['transfer_fee']

# Pearson correlation coefficient
pearson_coef, p_val = pearsonr(x, y)

print("Correlation between Market Value and Transfer Fees")
print(f"\t- Pearson correlation coefficient: {pearson_coef:.4f}")
print(f"\t- p-value (Pearson)              : {p_val:.4e}")
```

```
Correlation between Market Value and Transfer Fees
        - Pearson correlation coefficient: 0.9345
        - p-value (Pearson)              : 4.0218e-08
```

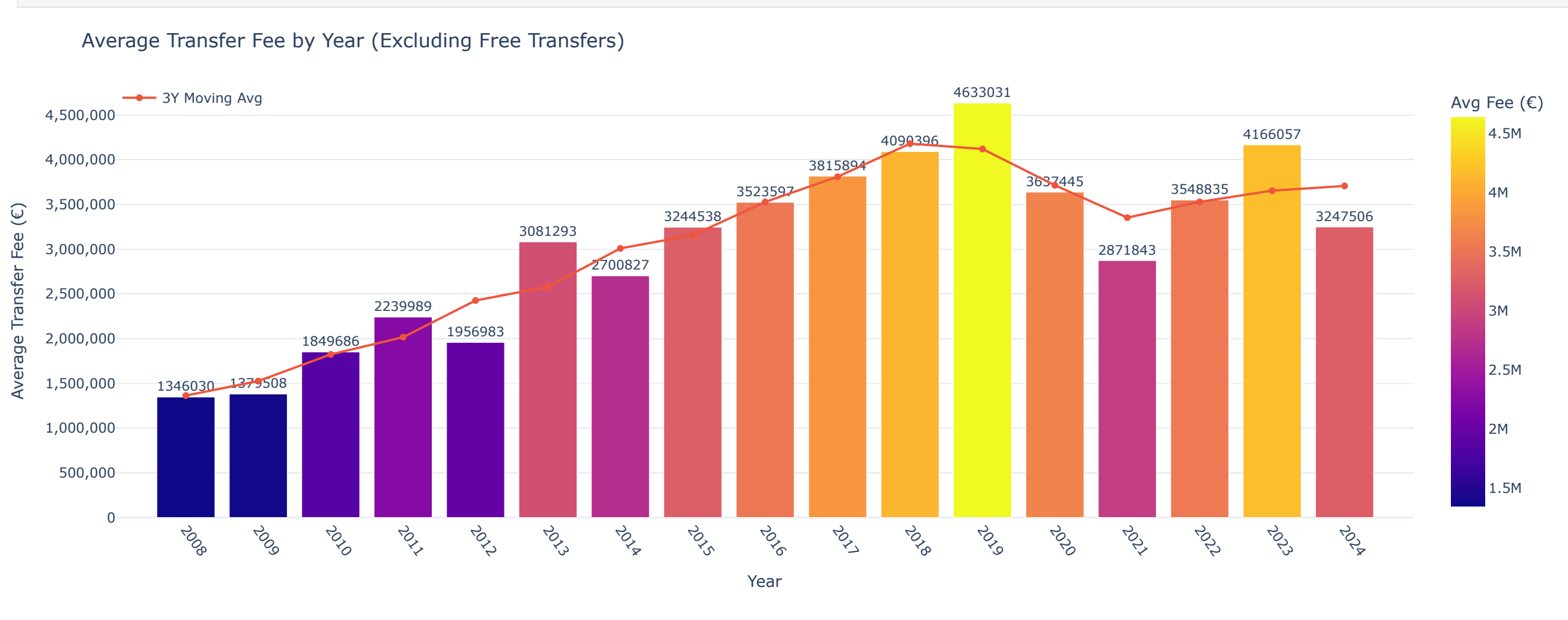
Pearson's correlation coefficient (0.9345) indicates a very strong and positive relationship between the average market value of players and the average transfer fees paid in transfers (excluding free transfers). In other words, as market value increases, so do transfer fees consistently.

The p-value (≈4.0e-08) is extremely low, confirming that this relationship is not random but statistically significant.

In practical terms: the transfer market responds closely to player valuations, validating that market value is a good predictor of actual transfer fees.

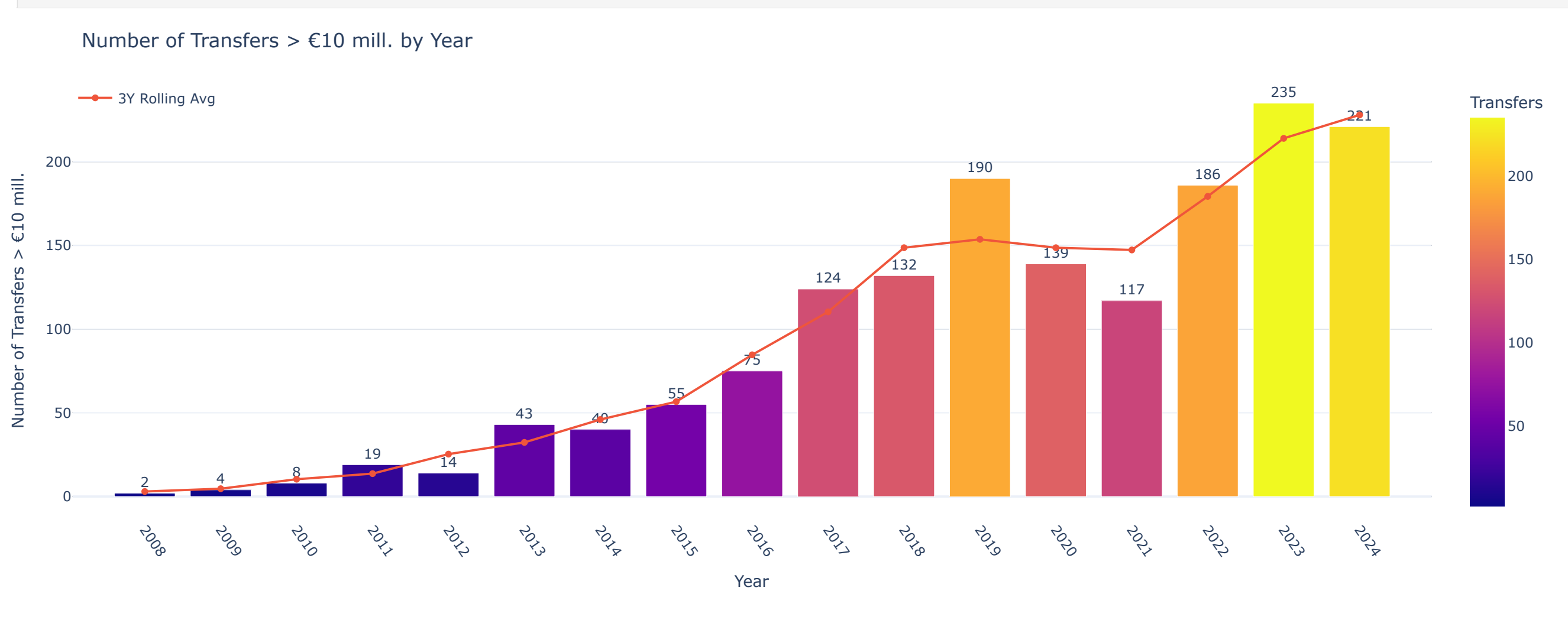
3.6. Evalution of Average Spend by Transfer by Year

In [22]:
```
# Base: year, cleaning and aggregations
df_f = (
    df_transfers_combined
      .dropna(subset=['transfer_fee', 'transfer_date'])
      .loc[df_transfers_combined['transfer_fee'] > 0]
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'] >= 2008]
)

yearly = (
    df_f.groupby('year', as_index=False)
        .agg(avg_fee=('transfer_fee', 'mean'))
        .sort_values('year')
)

# 3-year rolling average (centered to smooth)
yearly['roll3'] = (
    yearly['avg_fee']
    .rolling(3, center=True, min_periods=2)
    .mean()
)

# Graph
fig = px.bar(
    yearly,
    x='year',
    y='avg_fee',
    color='avg_fee',
    text=yearly['avg_fee'].round(0).astype(int),
    title='Average Transfer Fee by Year (Excluding Free Transfers)',
    hover_data={'year': True, 'avg_fee': ':.0f'}
)

fig.update_traces(textposition='outside', cliponaxis=False)

# Add trend line (3Y MA)
fig.add_trace(go.Scatter(
    x=yearly['year'],
    y=yearly['roll3'],
    mode='lines+markers',
    name='3Y Moving Avg',
    hovertemplate='Year: %{x}<br>3Y Avg: €%{y:,.0f}<extra></extra>'
))

# Style
fig.update_layout(
    legend=dict(
        orientation='v',
        yanchor='top',
        y=1,
        xanchor='left',
        x=0
    ),
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Average Transfer Fee (€)',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=yearly['year'],
        ticktext=yearly['year'],
        tickangle=55,
        rangeslider=dict(visible=False),
        rangeselector=dict(visible=False)
    ),
    coloraxis_colorbar=dict(title='Avg Fee (€)')
)

fig.update_yaxes(tickformat=',.0f')

fig.show()

fig.write_image(
    image_path / "avg_transfer_fee_by_year.png",
    width=1280,
    height=720,
    scale=2
)
```



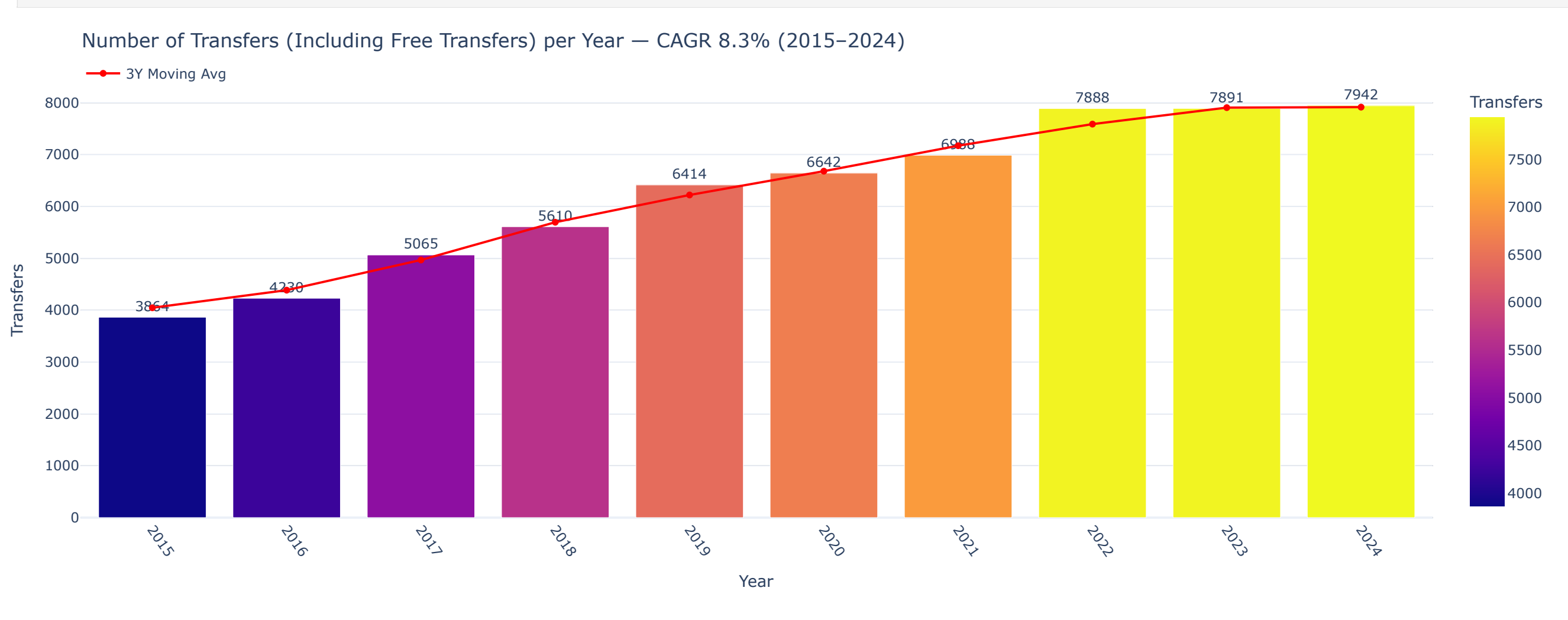
3.7. Number of Transfers of more than 10 mill. € by Year

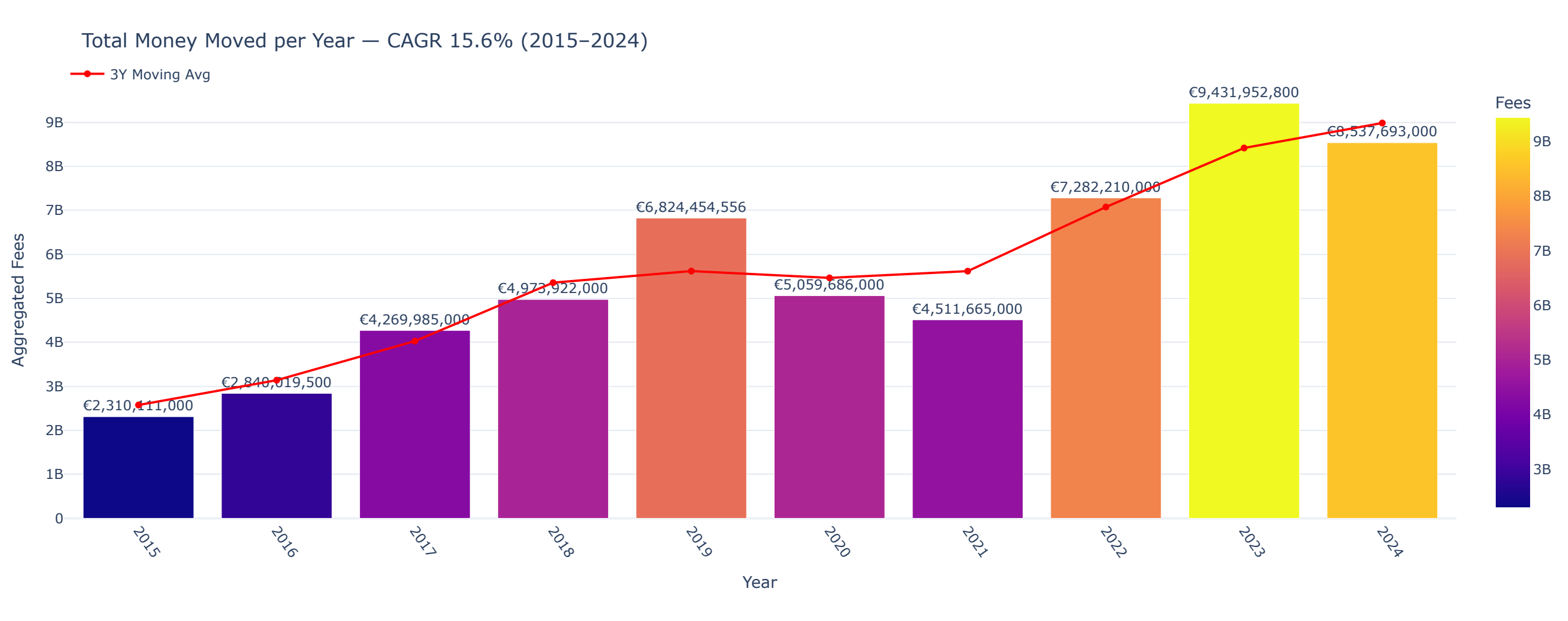
In [23]:
```
# Filter transfers over 10M from 2008
df_over_10m = (
    df_transfers_combined
      .dropna(subset=['transfer_fee', 'transfer_date'])
      .loc[df_transfers_combined['transfer_fee'] > 10_000_000]
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'] >= 2008]
)

# Count by year
transfers_over_10m_by_year = (
    df_over_10m.groupby('year', as_index=False)
              .agg(n_transfers=('transfer_fee', 'size'))
              .sort_values('year')
)

# Calculate 3-year rolling average
transfers_over_10m_by_year['roll3'] = (
    transfers_over_10m_by_year['n_transfers']
    .rolling(3, center=True, min_periods=2)
    .mean()
)

# Bar graph with continuous colors
fig = px.bar(
    transfers_over_10m_by_year,
    x='year',
    y='n_transfers',
    color='n_transfers',  # <- color continuo según valor
    text='n_transfers',
    title='Number of Transfers > €10 mill. by Year',
    hover_data={'year': True}
)
fig.update_traces(textposition='outside', cliponaxis=False)

# Add trend line
fig.add_trace(go.Scatter(
    x=transfers_over_10m_by_year['year'],
    y=transfers_over_10m_by_year['roll3'],
    mode='lines+markers',
    name='3Y Rolling Avg',
    hovertemplate='Year: %{x}<br>3Y Moving Avg: %{y:.0f}<extra></extra>'
))

# Style the graph
fig.update_layout(
    legend=dict(
        orientation='v',
        yanchor='top',
        y=1,
        xanchor='left',
        x=0
    ),
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Number of Transfers > €10 mill.',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=transfers_over_10m_by_year['year'],
        ticktext=transfers_over_10m_by_year['year'],
        tickangle=55,
        rangeslider=dict(visible=False),
        rangeselector=dict(visible=False)
    ),
    coloraxis_colorbar=dict(title='Transfers')
)

fig.show()

fig.write_image(
    image_path / "transfers_10M_per_year.png",
    width=1280,
    height=720,
    scale=2
)
```



3.8. Market Growth over the last 10 Years

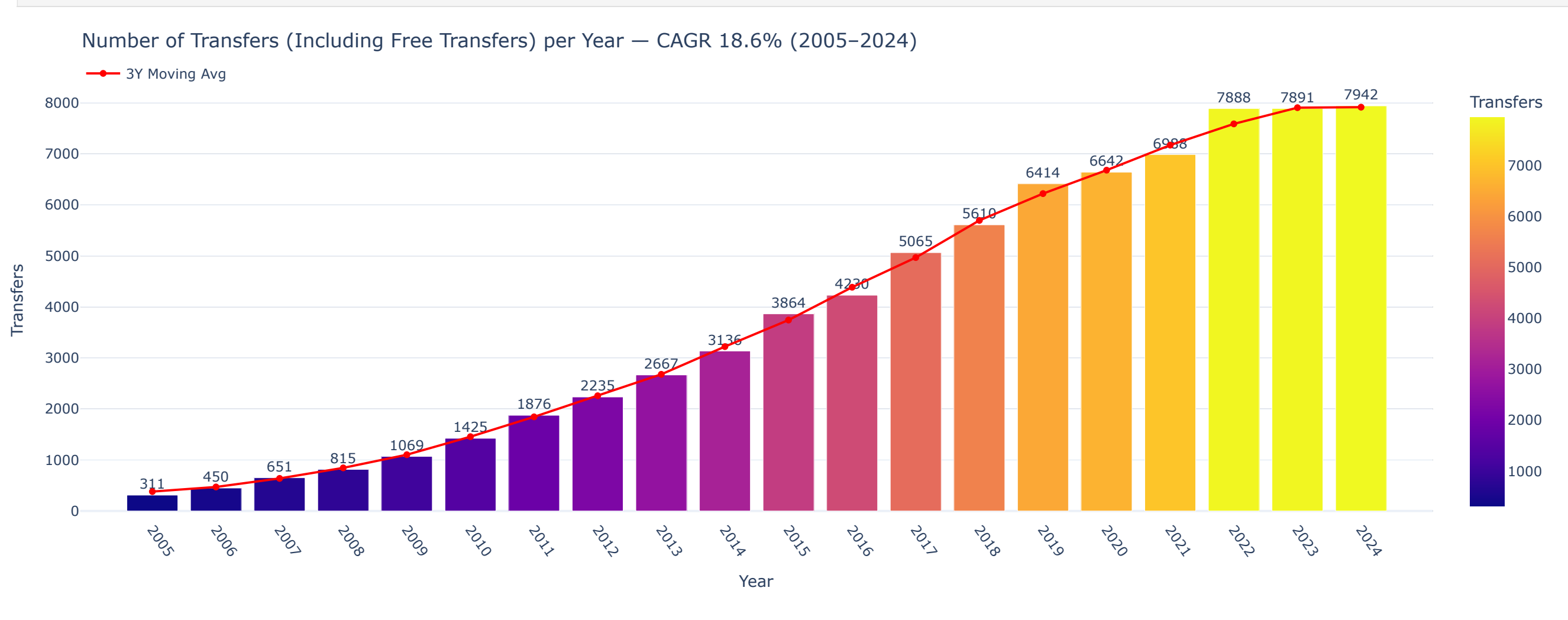
In [24]:
```
# Base: year, cleaning and aggregations
df_base = (
    df_transfers
      .dropna(subset=['transfer_date'])
      .assign(year=lambda d: d['transfer_date'].dt.year)
)

df_base2 = (
    df_transfers_combined
      .dropna(subset=['transfer_date'])
      .assign(year=lambda d: d['transfer_date'].dt.year)
)

# Count of transfers
counts = (
    df_base
      .groupby('year', as_index=True)
      .size()
      .rename('n_transfers')
      .sort_index()
)

# Money moved
money = (
    df_base2
      .dropna(subset=['transfer_fee'])
      .groupby('year', as_index=True)['transfer_fee']
      .sum()
      .rename('total_fee')
      .sort_index()
)

# Ensure continuity of years (fill with 0)
all_years = pd.Index(range(min(counts.index.min(), money.index.min()),
                           max(counts.index.max(), money.index.max())+1), name='year')
counts = counts.reindex(all_years, fill_value=0)
money  = money.reindex(all_years,  fill_value=0)

# Window: last 10 years available
end_year = int(all_years.max())
start_year = max(int(all_years.min()), end_year - 9)

counts_10 = counts.loc[start_year:end_year]
money_10  = money.loc[start_year:end_year]

years_diff = end_year - start_year  # number of years between extremes (for CAGR)

def safe_cagr(end_val, start_val, years):
    if years <= 0:
        return np.nan
    if start_val <= 0:
        return np.nan  # CAGR not defined if the start is 0 or negative
    return (end_val / start_val) ** (1/years) - 1

cagr_counts = safe_cagr(counts_10.iloc[-1], counts_10.iloc[0], years_diff)
cagr_money  = safe_cagr(money_10.iloc[-1],  money_10.iloc[0],  years_diff)

# 3-year moving average (centrada, permite 2 puntos al inicio/fin)
counts_10_roll3 = counts_10.rolling(3, center=True, min_periods=2).mean()
money_10_roll3  = money_10.rolling(3, center=True, min_periods=2).mean()


# Graph 1: Number of transfers + MA
fig1 = px.bar(
    counts_10.reset_index(),
    x='year',
    y='n_transfers',
    color='n_transfers',
    text='n_transfers',
    title=f'Number of Transfers (Including Free Transfers) per Year — CAGR {("" if pd.notna(cagr_counts) else "N/A") if pd.isna(cagr_counts) else f"{cagr_counts*100:.1f}%"} ({st
    hover_data={'year': True}
)
fig1.update_traces(textposition='outside', cliponaxis=False)

# Add trend line (3Y MA)
fig1.add_trace(go.Scatter(
    x=counts_10_roll3.index,
    y=counts_10_roll3,
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y Avg: %{y:,.0f}<extra></extra>'
))

fig1.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Transfers',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=counts_10.index,
        ticktext=counts_10.index.astype(str),
        tickangle=55
    ),
    coloraxis_colorbar=dict(title='Transfers'),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=0.98,
        xanchor='left',
        x=0
    )
)

fig1.show()

fig1.write_image(
    image_path / "transfers_per_year_CAGR.png",
    width=1280,
    height=720,
    scale=2
)


# Graph 2: Money moved + MA
fig2 = px.bar(
    money_10.reset_index(),
    x='year',
    y='total_fee',
    color='total_fee',
    text=money_10.reset_index()['total_fee'].map(lambda v: f"€{v:,.0f}"),
    title=f'Total Money Moved per Year — CAGR {("" if pd.notna(cagr_money) else "N/A") if pd.isna(cagr_money) else f"{cagr_money*100:.1f}%"} ({start_year}–{end_year})',
    hover_data={'year': True}
)
fig2.update_traces(textposition='outside', cliponaxis=False)

# Add trend line (3Y MA)
fig2.add_trace(go.Scatter(
    x=money_10_roll3.index,
    y=money_10_roll3,
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y Avg: €%{y:,.0f}<extra></extra>'
))

fig2.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Aggregated Fees',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=money_10.index,
        ticktext=money_10.index.astype(str),
        tickangle=55
    ),
    coloraxis_colorbar=dict(title='Fees'),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=0.98,
        xanchor='left',
        x=0
    )
)

fig2.show()

fig2.write_image(
    image_path / "money_moved_per_year_CAGR.png",
    width=1280,
    height=720,
    scale=2
)
```

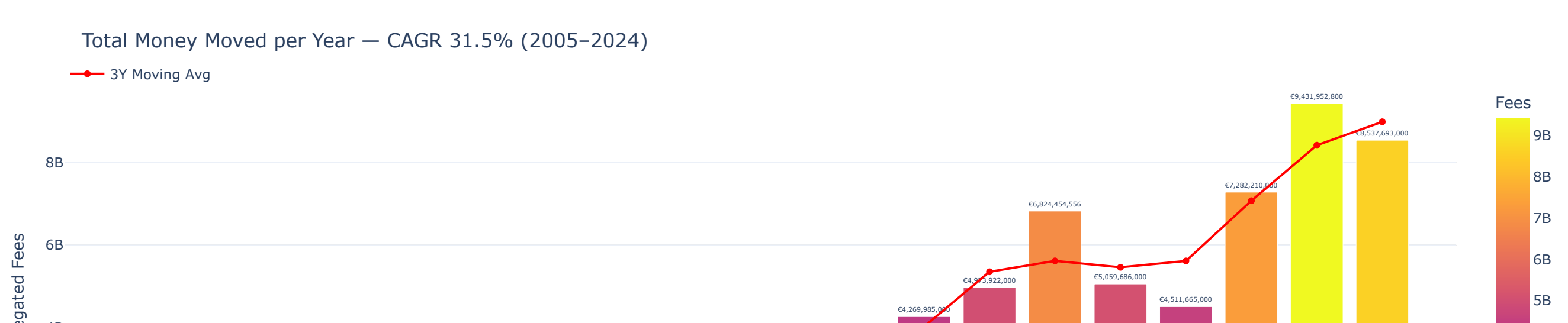


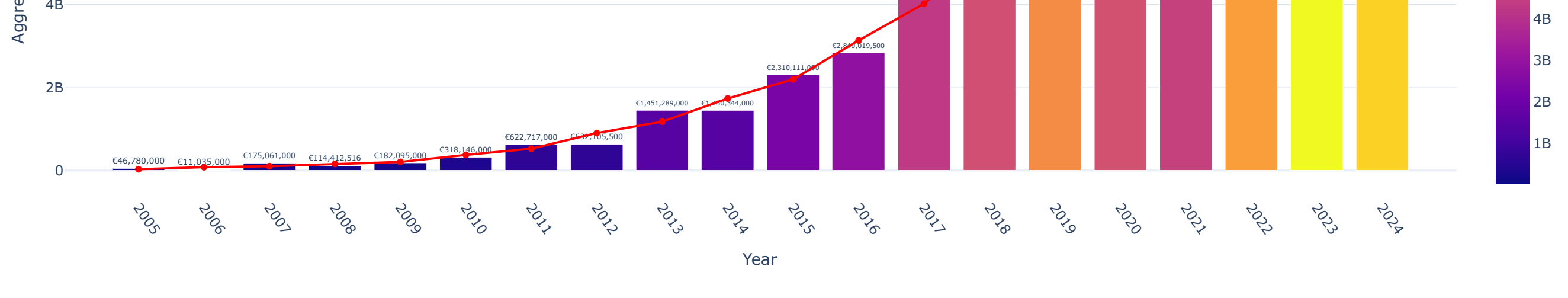


3.9. Market Growth over the last 20 Years

In [25]:
```
# Window: from 2005 (or the first available year if later)
start_2005 = max(2005, int(all_years.min()))
end_year   = int(all_years.max())

counts_2005 = counts.loc[start_2005:end_year]
money_2005  = money.loc[start_2005:end_year]

years_diff_2005 = end_year - start_2005

cagr_counts_2005 = safe_cagr(counts_2005.iloc[-1], counts_2005.iloc[0], years_diff_2005)
cagr_money_2005  = safe_cagr(money_2005.iloc[-1],  money_2005.iloc[0],  years_diff_2005)

# 3-year moving average (centered, allows 2 points at the beginning/end)
counts_2005_roll3 = counts_2005.rolling(3, center=True, min_periods=2).mean()
money_2005_roll3  = money_2005.rolling(3, center=True, min_periods=2).mean()

# Graph from 2005: Number of transfers
fig1_2005 = px.bar(
    counts_2005.reset_index(),
    x='year',
    y='n_transfers',
    color='n_transfers',
    text='n_transfers',
    title=(
        f'Number of Transfers (Including Free Transfers) per Year — CAGR '
        f'{("" if pd.notna(cagr_counts_2005) else "N/A") if pd.isna(cagr_counts_2005) else f"{cagr_counts_2005*100:.1f}%"} '
        f'({start_2005}–{end_year})'
    ),
    hover_data={'year': True}
)
fig1_2005.update_traces(textposition='outside', cliponaxis=False)

# Trend line (3Y MA)
fig1_2005.add_trace(go.Scatter(
    x=counts_2005_roll3.index,
    y=counts_2005_roll3,
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y Avg: %{y:,.0f}<extra></extra>'
))

fig1_2005.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Transfers',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=counts_2005.index,
        ticktext=counts_2005.index.astype(str),
        tickangle=55
    ),
    coloraxis_colorbar=dict(title='Transfers'),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=0.98,
        xanchor='left',
        x=0
    )
)

fig1_2005.show()

fig1_2005.write_image(
    image_path / "transfers_per_year_since2005_CAGR.png",
    width=1280,
    height=720,
    scale=2
)

# Graph from 2005: Money moved
fig2_2005 = px.bar(
    money_2005.reset_index(),
    x='year',
    y='total_fee',
    color='total_fee',
    text=money_2005.reset_index()['total_fee'].map(lambda v: f"€{v:,.0f}"),
    title=(
        f'Total Money Moved per Year — CAGR '
        f'{("" if pd.notna(cagr_money_2005) else "N/A") if pd.isna(cagr_money_2005) else f"{cagr_money_2005*100:.1f}%"} '
        f'({start_2005}–{end_year})'
    ),
    hover_data={'year': True}
)
fig2_2005.update_traces(textposition='outside', cliponaxis=False)

# Trend line (3Y MA)
fig2_2005.add_trace(go.Scatter(
    x=money_2005_roll3.index,
    y=money_2005_roll3,
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y Avg: €%{y:,.0f}<extra></extra>'
))

fig2_2005.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Aggregated Fees',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=money_2005.index,
        ticktext=money_2005.index.astype(str),
        tickangle=55
    ),
    coloraxis_colorbar=dict(title='Fees'),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=0.98,
        xanchor='left',
        x=0
    )
)

fig2_2005.show()

fig2_2005.write_image(
    image_path / "money_moved_per_year_since2005_CAGR.png",
    width=1280,
    height=720,
    scale=2
)
```



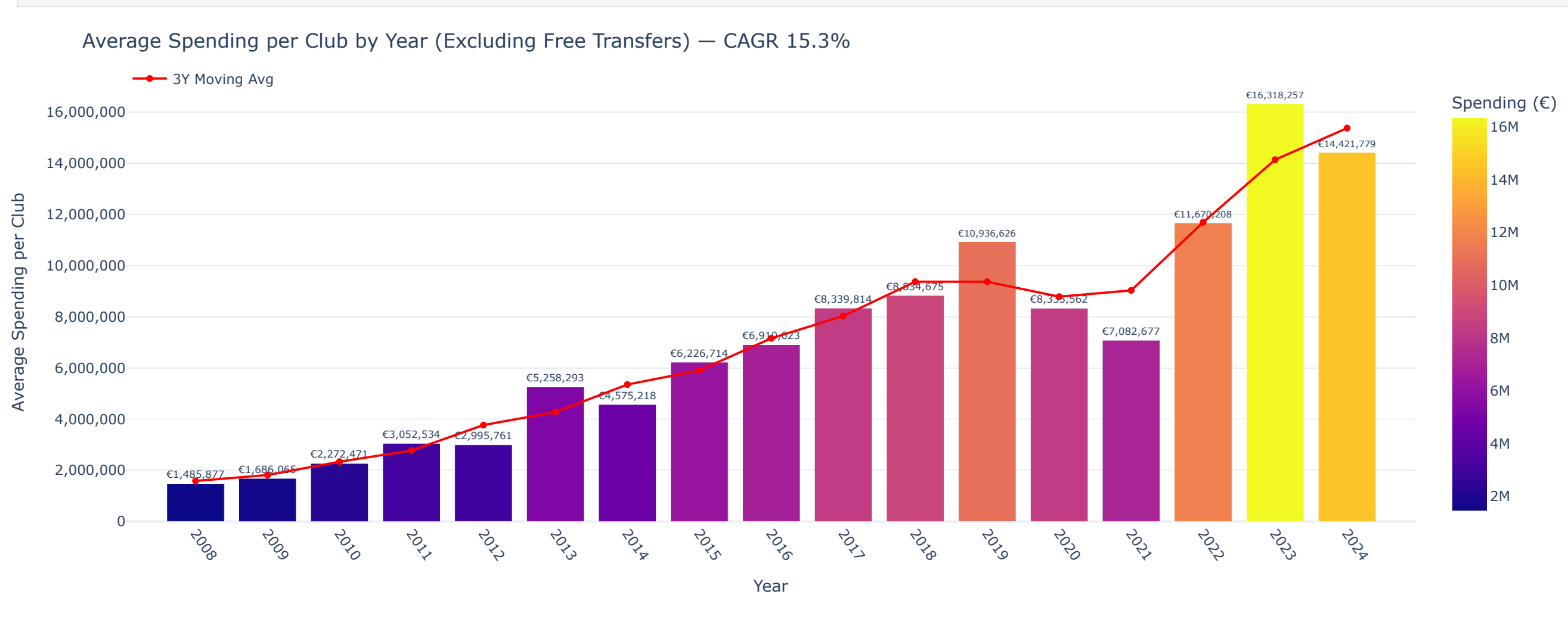




4. Club Analysis

4.1. Average Spending by Club

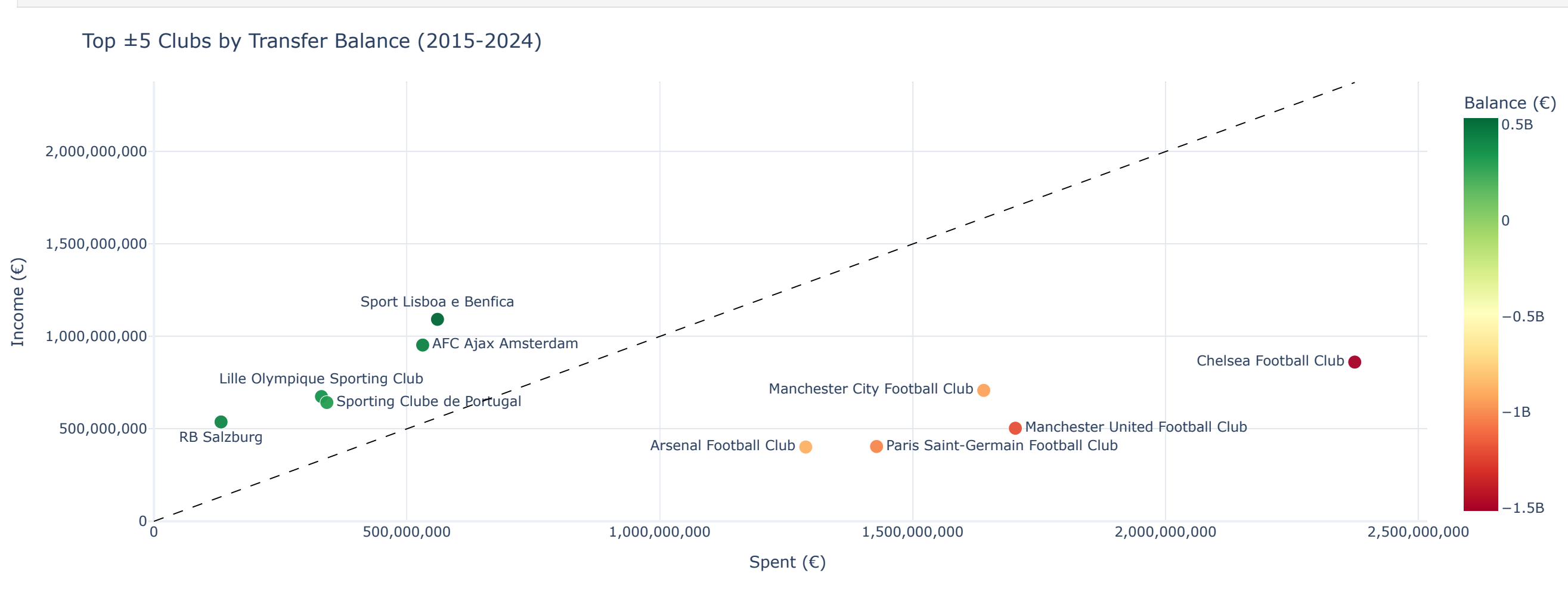
In [26]:
```python
# Filtering
df_club_spending = (
    df_transfers_combined
      .dropna(subset=['transfer_fee', 'transfer_date', 'to_club_id'])
      .loc[lambda d: d['transfer_fee'] > 0]
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'] >= 2008]
)

# Total spending per club and year
club_year_spending = (
    df_club_spending
      .groupby(['year', 'to_club_id'], as_index=False)
      .agg(total_spent=('transfer_fee', 'sum'))
)

# Average spending per club per year
avg_spending_per_club = (
    club_year_spending
      .groupby('year', as_index=False)
      .agg(avg_spent_per_club=('total_spent', 'mean'))
)

# Calculate CAGR
start_year = int(avg_spending_per_club['year'].min())
end_year   = int(avg_spending_per_club['year'].max())
years_diff = end_year - start_year
start_val  = avg_spending_per_club.loc[avg_spending_per_club['year'] == start_year, 'avg_spent_per_club'].iloc[0]
end_val    = avg_spending_per_club.loc[avg_spending_per_club['year'] == end_year, 'avg_spent_per_club'].iloc[0]

if start_val > 0:
    cagr = (end_val / start_val) ** (1 / years_diff) - 1
else:
    cagr = np.nan

# Order and MA(3)
avg_spending_per_club = avg_spending_per_club.sort_values('year').reset_index(drop=True)
avg_spending_per_club['roll3'] = (
    avg_spending_per_club['avg_spent_per_club']
    .rolling(3, center=True, min_periods=2)
    .mean()
)

# Bar graph
fig = px.bar(
    avg_spending_per_club,
    x='year',
    y='avg_spent_per_club',
    color='avg_spent_per_club',
    text=avg_spending_per_club['avg_spent_per_club'].map(lambda v: f"€{v:,.0f}"),
    title=f'Average Spending per Club by Year (Excluding Free Transfers) — CAGR {("" if pd.notna(cagr) else "N/A") if pd.isna(cagr) else f"{cagr*100:.1f}%"}',
    hover_data={'year': True}
)
fig.update_traces(textposition='outside', cliponaxis=False)

# Add trend line (3Y MA)
fig.add_trace(go.Scatter(
    x=avg_spending_per_club['year'],
    y=avg_spending_per_club['roll3'],
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y Avg: €%{y:,.0f}<extra></extra>'
))

# Style and legend above (clickable)
fig.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Average Spending per Club',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=avg_spending_per_club['year'],
        ticktext=avg_spending_per_club['year'],
        tickangle=55
    ),
    coloraxis_colorbar=dict(title='Spending (€)'),
    legend=dict(orientation='h', yanchor='bottom', y=0.97, xanchor='left', x=0)
)
fig.update_yaxes(tickformat=',.0f')

fig.show()

fig.write_image(
    image_path / "spending_per_club_by_year.png",
    width=1280,
    height=720,
    scale=2
)
```


Average Spending per Club by Year (Excluding Free Transfers) — CAGR 15.3%

4.2. Most and Least Spending Clubs of the last 10 Years

First, it is necessary to calculate the different aggregations that will be needed to produce the desired graph.

In [27]:
```python
# Last 10 years filter
last_year  = df_transfers_combined['transfer_date'].dt.year.max()
start_year = last_year - 9

df_last10 = (
    df_transfers_combined
      .dropna(subset=['transfer_fee','transfer_date'])
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'].between(start_year, last_year)]
)

# Club-level spending and income (sum of fees as buyer vs seller)
club_spending = (
    df_last10.groupby('to_club_id', as_index=False)['transfer_fee']
             .sum().rename(columns={'transfer_fee':'spent'})
)
club_income = (
    df_last10.groupby('from_club_id', as_index=False)['transfer_fee']
             .sum().rename(columns={'from_club_id':'club_id','transfer_fee':'income'})
)
club_spending = club_spending.rename(columns={'to_club_id':'club_id'})

# Merge spending & income and compute balance
club_balance = club_spending.merge(club_income, on='club_id', how='outer').fillna(0)
club_balance['balance'] = club_balance['income'] - club_balance['spent']

# Attach club names and competition
club_balance = club_balance.merge(
    df_clubs[['club_id','name','domestic_competition_id']],
    on='club_id', how='left'
)

# Competition-level totals (optional, kept for later use)
comp_balance = (
    club_balance.groupby('domestic_competition_id', as_index=False)
                .agg(total_spent=('spent','sum'),
                     total_income=('income','sum'))
)
comp_balance['balance'] = comp_balance['total_income'] - comp_balance['total_spent']
comp_balance = comp_balance.merge(
    df_competitions[['competition_id','name']],
    left_on='domestic_competition_id',
    right_on='competition_id',
    how='left'
)

# Clean numeric types and remove clubs with no signal (spent=0 and income=0)
club_balance['spent']  = pd.to_numeric(club_balance['spent'], errors='coerce').fillna(0.0)
club_balance['income'] = pd.to_numeric(club_balance['income'], errors='coerce').fillna(0.0)
plot_df = club_balance[(club_balance['spent'] > 0) | (club_balance['income'] > 0)].copy()

# Build fallback map from transfer data (most frequent observed name per club_id)
fallback_map = (
    pd.concat([
        df_transfers_combined[['to_club_id','to_club_name']].rename(columns={'to_club_id':'club_id','to_club_name':'club_name'}),
        df_transfers_combined[['from_club_id','from_club_name']].rename(columns={'from_club_id':'club_id','from_club_name':'club_name'})
    ], ignore_index=True)
    .dropna(subset=['club_id','club_name'])
    .groupby('club_id')['club_name']
    .agg(lambda s: s.mode().iloc[0])
)

# Final label: official name -> fallback from transfers -> numeric ID
plot_df['label'] = (
    plot_df['name']
      .fillna(plot_df['club_id'].map(fallback_map))
      .fillna(plot_df['club_id'].astype(str))
)

# Select Top 5 most profitable and Top 5 least profitable clubs
top_pos = plot_df.nlargest(5, 'balance')
top_neg = plot_df.nsmallest(5, 'balance')
top10   = pd.concat([top_pos, top_neg], ignore_index=True).drop_duplicates(subset='club_id')

# If there’s nothing to plot, print a helpful message
if top10.empty:
    print(f"No clubs with spending or income > 0 between {start_year}–{last_year}.")
else:
    # Diagonal extent
    max_axis = float(max(top10['spent'].max(), top10['income'].max()))
    max_axis = max_axis if max_axis > 0 else 1.0

    # Scatter: Top ±5 by balance, with labels (use 'label'!)
    fig = px.scatter(
        top10,
        x='spent',
        y='income',
        color='balance',
        text='label',
        hover_name='label',
        hover_data={
            'spent':':,.0f',
            'income':':,.0f',
            'balance':':,.0f',
            'club_id':True
        },
        color_continuous_scale='RdYlGn',
        title=f"Top ±5 Clubs by Transfer Balance ({start_year}–{last_year})"
    )

    positions = ['top center','middle right','bottom center','top center','middle right','middle left','middle right','middle right','middle left','middle left']

    fig.update_traces(
        marker=dict(size=12, opacity=0.95, line=dict(width=0.6, color='white')),
        textposition=positions * (len(top10)//len(positions) + 1)
    )

    # Equilibrium diagonal (income = spent)
    fig.add_shape(
        type='line',
        x0=0, y0=0,
        x1=max_axis, y1=max_axis,
        line=dict(color='black', dash='dash', width=1)
    )

    fig.update_layout(
        template='plotly_white',
        xaxis_title="Spent (€)",
        yaxis_title="Income (€)",
        hovermode='closest',
        margin=dict(l=40, r=20, t=60, b=80),
        coloraxis_colorbar=dict(title='Balance (€)'),
        legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='left', x=0)
    )

    fig.update_xaxes(tickformat=',.0f')
    fig.update_yaxes(tickformat=',.0f',scaleanchor=None, exponentformat="SI")
    fig.show()

    fig.write_image(
        image_path / "top_clubs_by_transfer_balance.png",
        width=1280,
        height=720,
        scale=2
    )
```


Top ±5 Clubs by Transfer Balance (2015-2024)

4.3. Percentage of Profitable Clubs by Year

It is important to know which clubs are profitable and which are not.

Profitable clubs are understood to be not only those with a positive transfer balance, but also those with a positive balance after subtracting the average salary expenditure for their league.

That is why, first of all, all the leagues that are included in the transfers will be obtained.

In [28]:
```python
# Base: filtering, cleaning and aggregating
df_base = (
    df_transfers_cleaned_both
      .dropna(subset=['transfer_fee', 'transfer_date'])
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'] >= 2008]
)

# Income by club-year (club as seller)
income_by_club_year = (
    df_base
      .groupby(['year', 'from_club_id'], as_index=False)
      .agg(total_income=('transfer_fee', 'sum'))
      .rename(columns={'from_club_id': 'club_id'})
)

# Spend by club-year (club as buyer)
spend_by_club_year = (
    df_base
      .groupby(['year', 'to_club_id'], as_index=False)
      .agg(total_spent=('transfer_fee', 'sum'))
      .rename(columns={'to_club_id': 'club_id'})
)

# Join income and spend
club_balance = (
    pd.merge(income_by_club_year, spend_by_club_year,
             on=['year', 'club_id'], how='outer')
      .fillna(0)
)

# Add competition to each club (taken from df_clubs, via club_id)
club_balance = club_balance.merge(
    df_clubs[['club_id', 'domestic_competition_id']],
    on='club_id', how='left'
)

# Optional diagnostic
missing_comp = club_balance['domestic_competition_id'].isna().sum()
print(f"Rows without domestic_competition_id after merge: {missing_comp}")

# List unique competitions present in club_balance
unique_comps = (
    club_balance[['domestic_competition_id']]
      .dropna()
      .drop_duplicates()
      .merge(
          df_competitions[['competition_id','name','country_name']],
          left_on='domestic_competition_id',
          right_on='competition_id',
          how='left'
      )
      .sort_values('competition_id')
      .reset_index(drop=True)
)

print("Total unique competitions in club_balance:", len(unique_comps))
display(unique_comps)
```

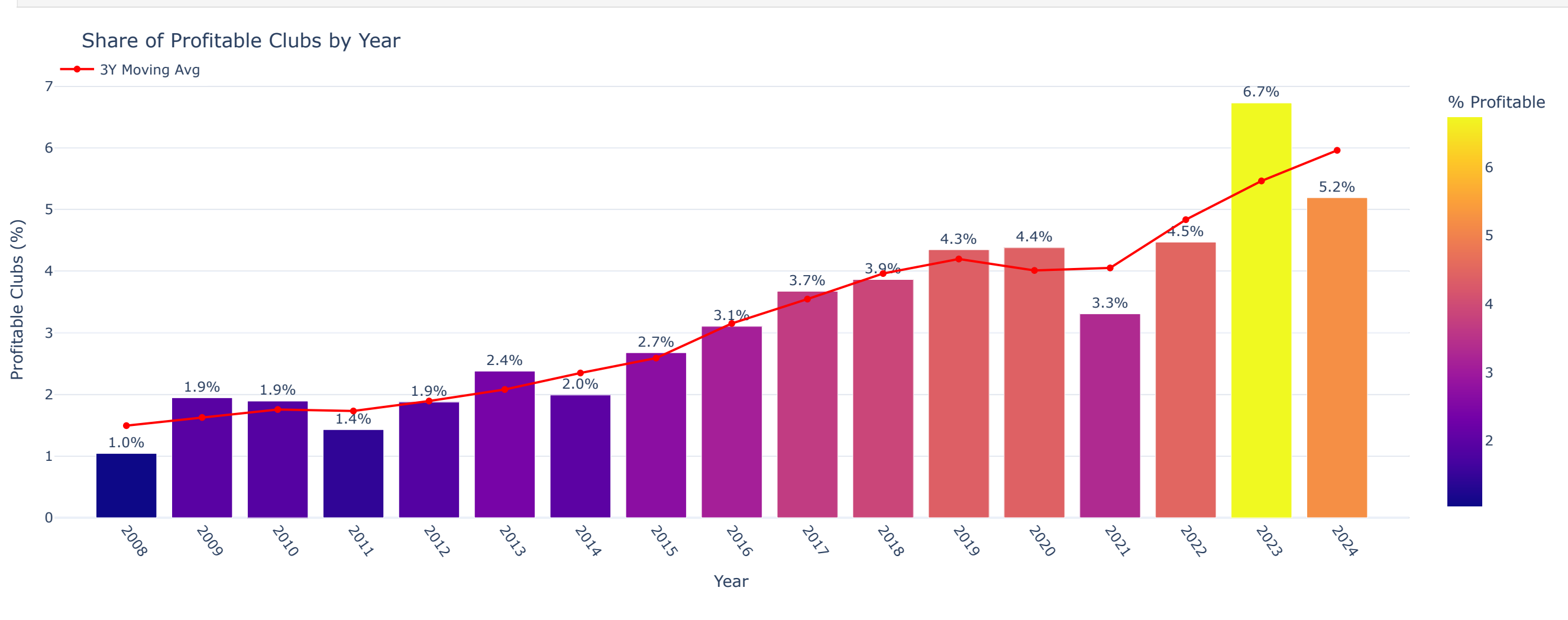
```
Rows without domestic_competition_id after merge: 11227
Total unique competitions in club_balance: 14
```

	domestic_competition_id	competition_id	name	country_name
0	BE1	BE1	jupiler-pro-league	Belgium
1	DK1	DK1	superligaen	Denmark
2	ES1	ES1	laliga	Spain
3	FR1	FR1	ligue-1	France
4	GB1	GB1	premier-league	England
5	GR1	GR1	super-league-1	Greece
6	IT1	IT1	serie-a	Italy
7	L1	L1	bundesliga	Germany
8	NL1	NL1	eredivisie	Netherlands
9	PO1	PO1	liga-portugal-bwin	Portugal
10	RU1	RU1	premier-liga	Russia
11	SC1	SC1	scottish-premiership	Scotland
12	TR1	TR1	super-lig	Turkey
13	UKR1	UKR1	premier-liga	Ukraine

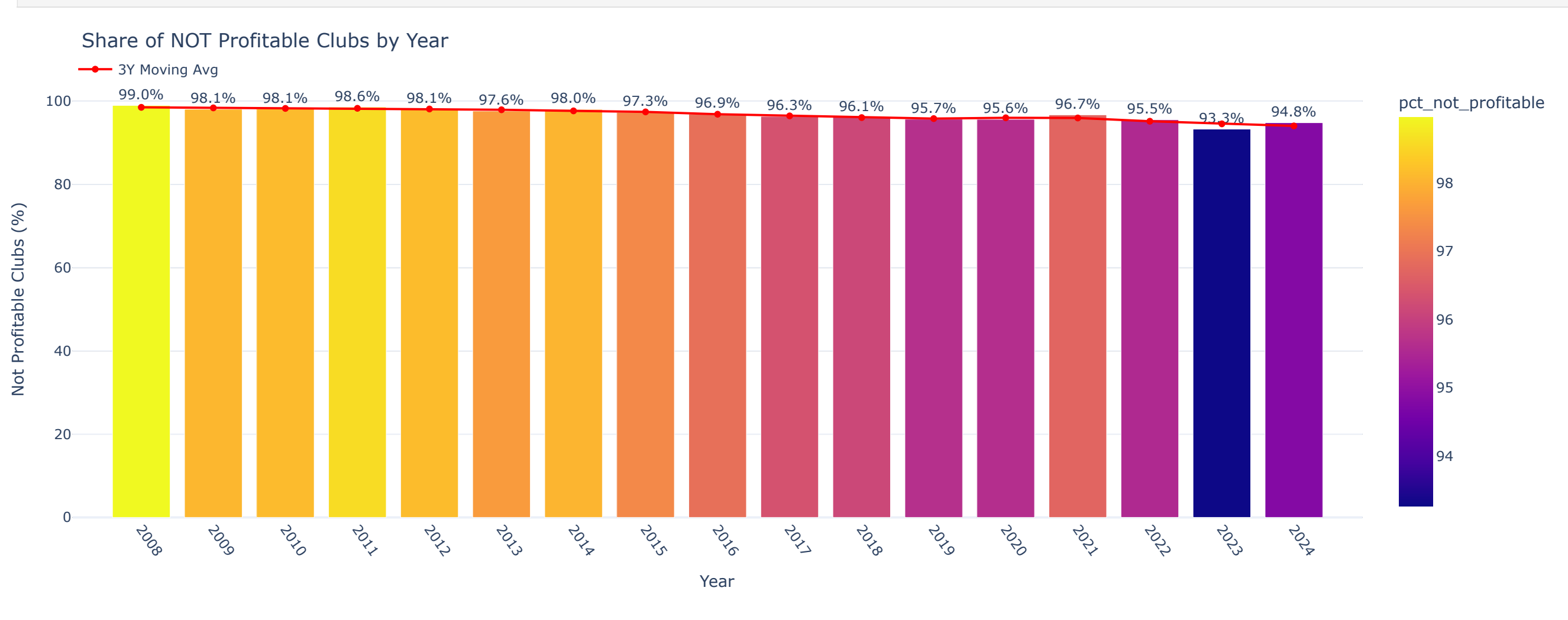
The average expenditure per club in each league, taken from various cited sources, will be selected and subtracted from the balance.

If there is no corresponding league, the lowest value among those from the verified sources will be subtracted.

In [29]:
```python
# Map league-average annual salary per club (EUR)
league_salary_avg_eur = {
    # Big 5
    'GB1': 158.04e6,  # Premier League
    'ES1':  97.92e6,  # LaLiga
    'IT1':  77.58e6,  # Serie A
    'L1' :  69.84e6,  # Bundesliga (your ID)
    'FR1':  66.87e6,  # Ligue 1
    # Others
    'TR1':  19.35e6,  # Turkey Super Lig
    'BE1':  16.47e6,  # Belgium Pro League
    'RU1':  18.45e6,  # Russia Premier Liga
    'PO1':  13.59e6,  # Portugal Liga
    'NL1':  12.60e6,  # Netherlands Eredivisie
    'SC1':   6.57e6,  # Scotland Premiership
    'DK1':   4.23e6,  # Denmark Superliga
}

# Fallback to the smallest defined league wage if a league is unknown
min_salary = min(league_salary_avg_eur.values())
club_balance['league_salary_eur'] = (
    club_balance['domestic_competition_id']
      .map(league_salary_avg_eur)
      .fillna(min_salary)
      .astype(float)
)

# Adjusted balance after wages (per club-year) and profitability flag
club_balance['adj_balance_after_wages'] = (
    club_balance['total_income'] - club_balance['total_spent'] - club_balance['league_salary_eur']
)
club_balance['profitable_after_wages'] = club_balance['adj_balance_after_wages'] > 0

# Yearly summary: % of clubs profitable after wages
profitability_by_year_wages = (
    club_balance
      .groupby('year', as_index=False)
      .agg(
          pct_profitable=('profitable_after_wages', lambda x: 100 * x.mean()),
          n_clubs=('club_id', 'nunique')
      )
      .sort_values('year')
      .reset_index(drop=True)
)

# 3Y Moving Average (choose centered or trailing)
profitability_by_year_wages['roll3'] = (
    profitability_by_year_wages['pct_profitable']
      .rolling(window=3, center=True, min_periods=2)
      .mean()
)

fig = px.bar(
    profitability_by_year_wages,
    x='year',
    y='pct_profitable',
    color='pct_profitable',
    text=profitability_by_year_wages['pct_profitable'].map(lambda v: f"{v:.1f}%"),
    title='Share of Profitable Clubs by Year',
    hover_data={'year': True, 'n_clubs': True}
)
fig.update_traces(textposition='outside', cliponaxis=False)

# Add red continuous 3Y MA line
fig.add_trace(go.Scatter(
    x=profitability_by_year_wages['year'],
    y=profitability_by_year_wages['roll3'],
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y MA: %{y:.1f}%<extra></extra>'
))

# Styling
fig.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Profitable Clubs (%)',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=profitability_by_year_wages['year'],
        ticktext=profitability_by_year_wages['year'],
        tickangle=55
    ),
    coloraxis_colorbar=dict(title='% Profitable'),
    legend=dict(orientation='h', yanchor='bottom', y=0.99, xanchor='left', x=0)
)
fig.show()

fig.write_image(
    image_path / "profitable_clubs_by_year.png",
    width=1280,
    height=720,
    scale=2
)
```

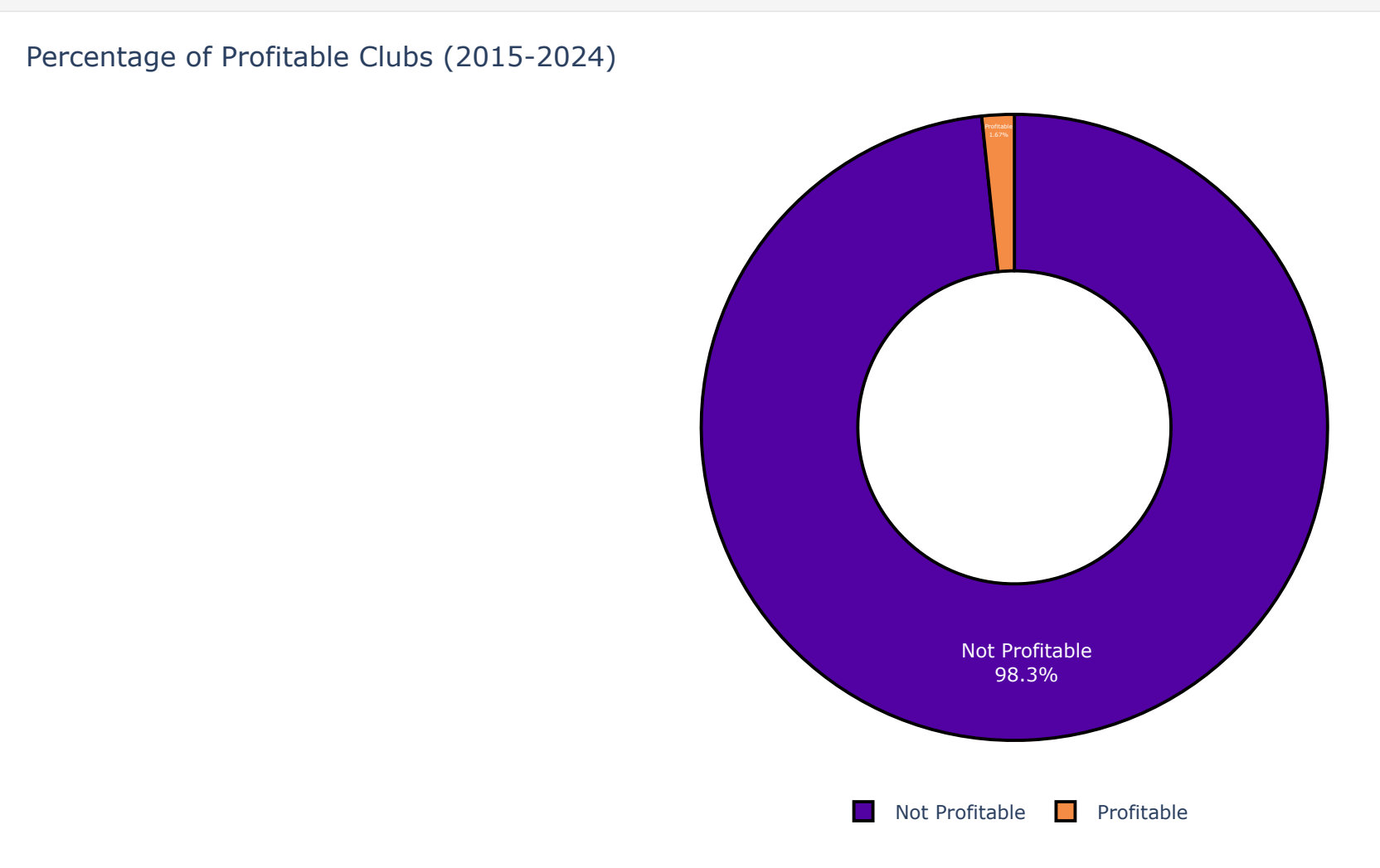

Share of Profitable Clubs by Year

In [30]:
```python
# % of clubs NOT profitable by year (inverse)
not_profitability_by_year_wages = (
    club_balance
      .groupby('year', as_index=False)
      .agg(
          pct_not_profitable=('profitable_after_wages', lambda x: 100 * (~x).mean()),
          n_clubs=('club_id', 'nunique')
      )
      .sort_values('year')
      .reset_index(drop=True)
)

# 3Y Moving Average (centered)
not_profitability_by_year_wages['roll3'] = (
    not_profitability_by_year_wages['pct_not_profitable']
      .rolling(window=3, center=True, min_periods=2)
      .mean()
)

fig = px.bar(
    not_profitability_by_year_wages,
    x='year',
    y='pct_not_profitable',
    color='pct_not_profitable',
    text=not_profitability_by_year_wages['pct_not_profitable'].map(lambda v: f"{v:.1f}%"),
    title='Share of NOT Profitable Clubs by Year',
    hover_data={'year': True, 'n_clubs': True}
)
fig.update_traces(textposition='outside', cliponaxis=False)

# Red line of MA(3)
fig.add_trace(go.Scatter(
    x=not_profitability_by_year_wages['year'],
    y=not_profitability_by_year_wages['roll3'],
    mode='lines+markers',
    name='3Y Moving Avg',
    line=dict(color='red', width=2),
    hovertemplate='Year: %{x}<br>3Y MA: %{y:.1f}%<extra></extra>'
))

# Style
fig.update_layout(
    template='plotly_white',
    xaxis_title='Year',
    yaxis_title='Not Profitable Clubs (%)',
    hovermode='x unified',
    margin=dict(l=40, r=20, t=60, b=80),
    xaxis=dict(
        tickmode='array',
        tickvals=not_profitability_by_year_wages['year'],
        ticktext=not_profitability_by_year_wages['year'],
        tickangle=55
    ),
    showlegend=True,
    legend=dict(orientation='h', yanchor='bottom', y=0.99, xanchor='left', x=0)
)

fig.show()

fig.write_image(
    image_path / "not_profitable_clubs_by_year.png",
    width=1280,
    height=720,
    scale=2
)
```


Share of NOT Profitable Clubs by Year

4.4. Percentage of Profitable Clubs of the last Decade

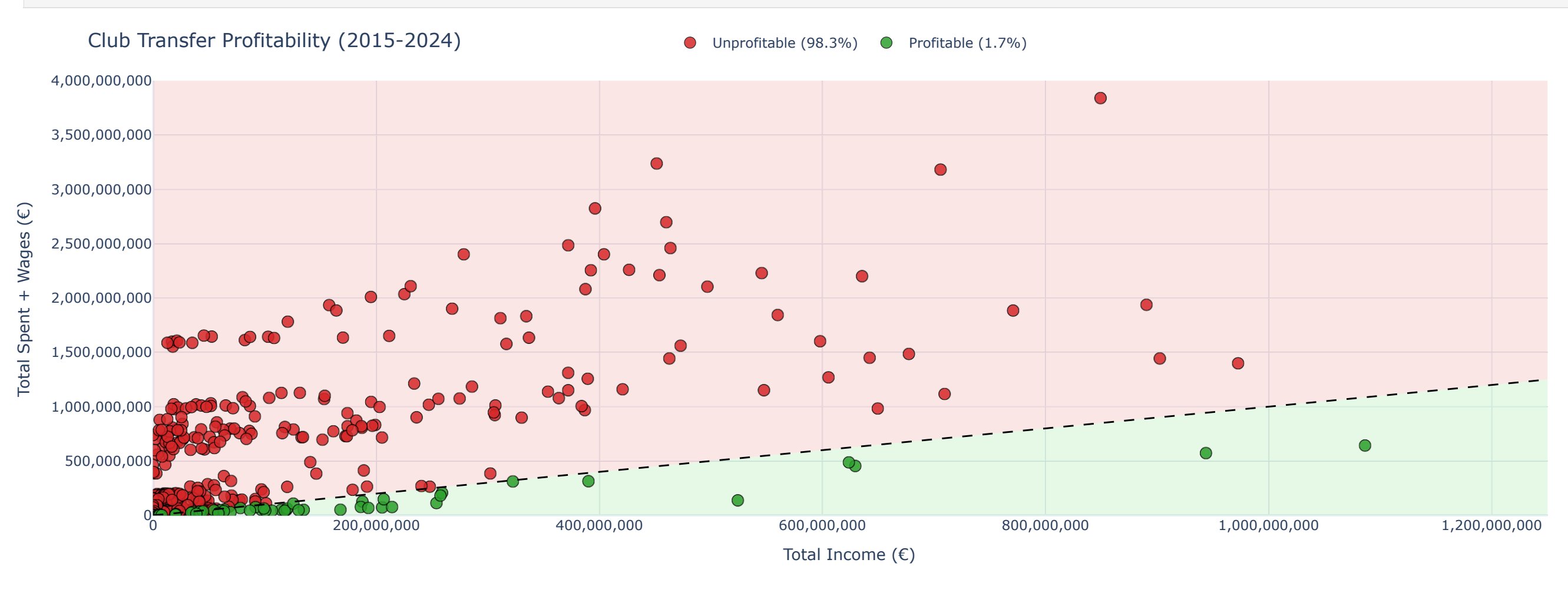
In [31]:
```python
# Last 10-year cumulative profitability donut (AFTER wages × active seasons)

# Define rolling 10-year window
last_year  = df_transfers_cleaned_both['transfer_date'].dt.year.max()
start_year = int(last_year) - 9

# Filter, clean, and keep only last 10 years
df_base = (
    df_transfers_cleaned_both
      .dropna(subset=['transfer_fee', 'transfer_date'])
      .assign(year=lambda d: d['transfer_date'].dt.year)
      .loc[lambda d: d['year'].between(start_year, last_year)]
)

# Cumulative transfer income per club (as seller)
income_by_club = (
    df_base
      .groupby('from_club_id', as_index=False)
      .agg(total_income=('transfer_fee', 'sum'))
      .rename(columns={'from_club_id': 'club_id'})
)

# Cumulative transfer spend per club (as buyer)
spend_by_club = (
    df_base
      .groupby('to_club_id', as_index=False)
      .agg(total_spent=('transfer_fee', 'sum'))
      .rename(columns={'to_club_id': 'club_id'})
)

# Merge to club-level balance (10Y window)
club_balance_total = (
    pd.merge(income_by_club, spend_by_club, on='club_id', how='outer')
      .fillna(0)
)

# Attach competition ID to each club
club_balance_total = club_balance_total.merge(
    df_clubs[['club_id', 'domestic_competition_id']],
    on='club_id', how='left'
)

# Mapping of average annual salary by club
league_salary_avg_eur = {
    # Big 5
    'GB1': 158.04e6,  # Premier League
    'ES1': 97.92e6,   # LaLiga
    'IT1': 77.58e6,   # Serie A
    'L1' : 69.84e6,   # Bundesliga
    'FR1': 66.87e6,   # Ligue 1

    # Others in the list
    'TR1': 19.35e6,   # Turkey Super Lig
    'BE1': 16.47e6,   # Belgium Pro League
    'RU1': 18.45e6,   # Russia Premier Liga
    'PO1': 13.59e6,   # Portugal Liga
    'NL1': 12.60e6,   # Netherlands Eredivisie
    'SC1': 6.57e6,    # Scotland Premiership
    'DK1': 4.23e6,    # Denmark Superliga
}
min_salary = min(league_salary_avg_eur.values())

club_balance_total['league_salary_eur'] = (
    club_balance_total['domestic_competition_id']
      .map(league_salary_avg_eur)
      .fillna(min_salary)
      .astype(float)
)

# ACTIVE SEASONS per club within the 10Y window (club appears either as buyer or seller)
club_year_presence = pd.concat([
    df_base[['year', 'from_club_id']].rename(columns={'from_club_id':'club_id'}),
    df_base[['year', 'to_club_id']].rename(columns={'to_club_id':'club_id'})
], ignore_index=True).dropna().drop_duplicates()

active_years = (
    club_year_presence
      .groupby('club_id', as_index=False)['year']
      .nunique()
      .rename(columns={'year':'active_seasons'})
)

club_balance_total = club_balance_total.merge(active_years, on='club_id', how='left')
club_balance_total['active_seasons'] = club_balance_total['active_seasons'].fillna(0).astype(int)

# Total wage cost over the window = league-average per year × active seasons
club_balance_total['wage_cost_10y'] = (
    club_balance_total['league_salary_eur'] * club_balance_total['active_seasons']
)

# Adjusted balance after wages (10Y) & profitability flag
club_balance_total['adj_balance_after_wages'] = (
    club_balance_total['total_income'] - club_balance_total['total_spent'] - club_balance_total['wage_cost_10y']
)
club_balance_total['profitable_after_wages'] = club_balance_total['adj_balance_after_wages'] > 0

# Donut values
values = [
    int(club_balance_total['profitable_after_wages'].sum()),
    int((~club_balance_total['profitable_after_wages']).sum())
]
labels = ['Profitable', 'Not Profitable']

fig = px.pie(
    names=labels,
    values=values,
    title=f'Percentage of Profitable Clubs ({start_year}–{last_year})',
    hole=0.5,
    color=labels,
    color_discrete_map={
        'Profitable': '#f48c45',
        'Not Profitable': '#5202a2'
    }
)

# Styling & labels
fig.update_traces(
    textinfo='label+percent',
    textposition='inside',
    insidetextorientation='horizontal',
    insidetextfont=dict(color='white'),
    outsidetextfont=dict(color='white'),
    texttemplate='%{label}<br>%{percent}',
    marker=dict(line=dict(color='#000000', width=2))
)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=-0.15,
        xanchor='center',
        x=0.5
    ),
    margin=dict(l=40, r=40, t=60, b=40)
)

fig.show()

fig.write_image(
    image_path / "profitability_by_decade.png",
    width=1280,
    height=720,
    scale=2
)
```


Percentage of Profitable Clubs (2015-2024)

4.5. Clustering of Profitable Clubs of the last Decade

In [32]:
```python
# Dataset to plot
plot_df = club_balance_total.merge(
    df_clubs[['club_id','name']],
    on='club_id', how='left'
).copy()

plot_df['total_spent_after_wages'] = (
    pd.to_numeric(plot_df['total_spent'], errors='coerce').fillna(0)
    + pd.to_numeric(plot_df['wage_cost_10y'], errors='coerce').fillna(0)
)

# Rename categories
plot_df['profit_label'] = plot_df['profitable_after_wages'].map({
```

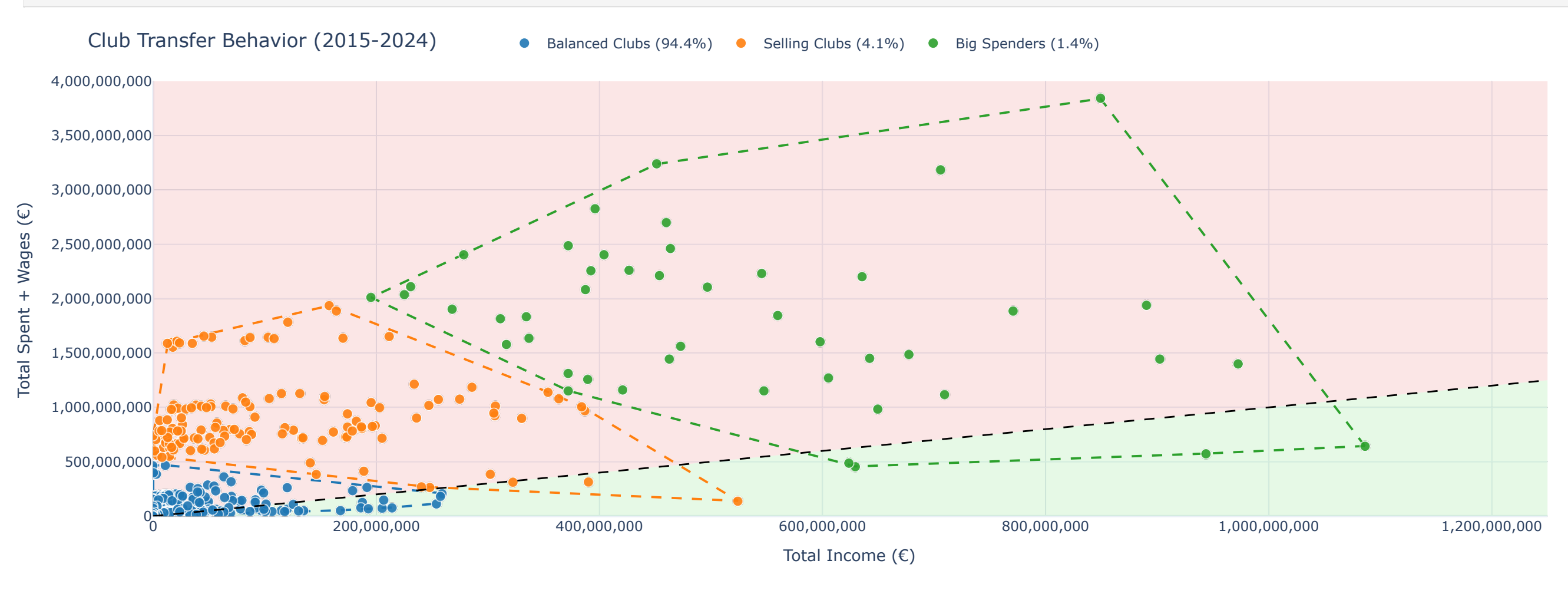
```
plot_df['profit_label'] = plot_df['profitable_after_wages'].map({
    True: 'Profitable',
    False: 'Unprofitable'
})

# Calculate percentages
counts = plot_df['profit_label'].value_counts(normalize=True) * 100
label_map = {
    'Profitable': f"Profitable ({counts.get('Profitable',0):.1f}%)",
    'Unprofitable': f"Unprofitable ({counts.get('Unprofitable',0):.1f}%)"
}
plot_df['profit_label_pct'] = plot_df['profit_label'].map(label_map)

# Limits
x_max = 1.25e9
y_max = 4e9

# Colored zones
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=[0, x_max, x_max, 0],
    y=[0, 0,     x_max, 0],
    fill='toself', mode='lines',
    line=dict(width=0),
    fillcolor='rgba(0,200,0,0.1)',
    hoverinfo='skip', showlegend=False
))

fig.add_trace(go.Scatter(
    x=[0, x_max, x_max, 0],
    y=[0, x_max, y_max, y_max],
    fill='toself', mode='lines',
    line=dict(width=0),
    fillcolor='rgba(220,0,0,0.1)',
    hoverinfo='skip', showlegend=False
))

# Colored points
scatter = px.scatter(
    plot_df,
    x='total_income',
    y='total_spent_after_wages',
    color='profit_label_pct',
    hover_name='name',
    hover_data={
        'total_income':':,.0f',
        'total_spent':':,.0f',
        'wage_cost_10y':':,.0f',
        'total_spent_after_wages':':,.0f',
        'adj_balance_after_wages':':,.0f',
        'active_seasons':True,
        'domestic_competition_id':True
    },
    color_discrete_map={
        f"Profitable ({counts.get('Profitable',0):.1f}%)": '#2ca02c',
        f"Unprofitable ({counts.get('Unprofitable',0):.1f}%)": '#d62728'
    }
)

scatter.update_traces(marker=dict(size=10, opacity=0.85,
                                  line=dict(width=0.6, color='black')))

for trace in scatter.data:
    fig.add_trace(trace)

# Equilibrium line
fig.add_shape(
    type='line',
    x0=0, y0=0, x1=x_max, y1=x_max,
    line=dict(color='black', dash='dash', width=1.5)
)

# Layout
fig.update_layout(
    title=f"Club Transfer Profitability ({start_year}–{last_year})",
    template='plotly_white',
    xaxis_title="Total Income (€)",
    yaxis_title="Total Spent + Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=1.05,
        xanchor='center',
        x=0.5,
        title=None
    )
)

# Limits
fig.update_xaxes(range=[0, x_max], tickformat=",.0f")
fig.update_yaxes(range=[0, y_max], tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "club_profitability_scatter_zones.png",
    width=1280, height=720, scale=2
)
```



4.6. Clustering of Type of Clubs of the last Decade

In [33]:

```
# Base data and metrics
plot_df = club_balance_total.merge(
    df_clubs[['club_id','name']],
    on='club_id', how='left'
).copy()

plot_df['total_spent_after_wages'] = (
    pd.to_numeric(plot_df['total_spent'], errors='coerce').fillna(0) +
    pd.to_numeric(plot_df['wage_cost_10y'], errors='coerce').fillna(0)
)

# Just clubs with some signal (income or spending)
plot_df = plot_df[(plot_df['total_income'] > 0) | (plot_df['total_spent_after_wages'] > 0)].copy()

# Limits
x_max = 1.25e9
y_max = 4e9

# K-Means (k=3) over scaled X,Y
X_scaled = np.column_stack([
    np.clip(plot_df['total_income'].values / x_max, 0, 1),
    np.clip(plot_df['total_spent_after_wages'].values / y_max, 0, 1)
])

kmeans = KMeans(n_clusters=3, n_init=25, random_state=42)
labels = kmeans.fit_predict(X_scaled)
centroids_scaled = kmeans.cluster_centers_

# Centroids in original scale
centroids = np.column_stack([centroids_scaled[:,0]*x_max, centroids_scaled[:,1]*y_max])

# Assign names to clusters
# Distance to origin in scaled space
r = np.sqrt((centroids_scaled[:,0])**2 + (centroids_scaled[:,1])**2)
idx_balanced = int(np.argmin(r))

# Of the two remaining, the one with the highest X = Big Spenders
rest = [i for i in range(3) if i != idx_balanced]
idx_big_spenders = rest[int(np.argmax(centroids[rest, 0]))]
idx_selling = [i for i in rest if i != idx_big_spenders][0]

cluster_name_map = {idx_balanced: 'Balanced Clubs',
                    idx_big_spenders: 'Big Spenders',
                    idx_selling: 'Selling Clubs'}

plot_df['cluster'] = [cluster_name_map[i] for i in labels]

# Percentages by group for legend
counts = plot_df['cluster'].value_counts()
total  = int(counts.sum())
pct = (counts / total * 100).round(1)
legend_labels = {
    'Balanced Clubs': f"Balanced Clubs ({pct.get('Balanced Clubs',0):.1f}%)",
    'Selling Clubs':  f"Selling Clubs ({pct.get('Selling Clubs',0):.1f}%)",
    'Big Spenders':   f"Big Spenders ({pct.get('Big Spenders',0):.1f}%)",
}

# Figure with zones
fig = go.Figure()

# Green zone (below y=x)
fig.add_trace(go.Scatter(
    x=[0, x_max, x_max, 0],
    y=[0, 0,     x_max, 0],
    fill='toself', mode='lines', line=dict(width=0),
    fillcolor='rgba(0,200,0,0.10)', hoverinfo='skip', showlegend=False
))
# Red zone (above y=x, all the way to the top)
fig.add_trace(go.Scatter(
    x=[0, x_max, x_max, 0],
    y=[0, x_max, y_max, y_max],
    fill='toself', mode='lines', line=dict(width=0),
    fillcolor='rgba(220,0,0,0.10)', hoverinfo='skip', showlegend=False
))

# Points by cluster
color_map = {
    'Balanced Clubs': '#1f77b4',
    'Selling Clubs':  '#ff7f0e',
    'Big Spenders':   '#2ca02c',
}

scatter = px.scatter(
    plot_df,
    x='total_income',
    y='total_spent_after_wages',
    color='cluster',
    color_discrete_map=color_map,
    hover_name='name',
    hover_data={
        'total_income':':,.0f',
        'total_spent':':,.0f',
        'wage_cost_10y':':,.0f',
        'total_spent_after_wages':':,.0f'
    }
)
scatter.update_traces(marker=dict(size=9, opacity=0.9, line=dict(width=0.6, color='white')))

# Rename series in legend with corresponding %
for tr in scatter.data:
    tr.name = legend_labels.get(tr.name, tr.name)
    fig.add_trace(tr)

# Equilibrium line
fig.add_shape(type='line', x0=0, y0=0, x1=x_max, y1=x_max,
              line=dict(color='black', dash='dash', width=1.5))

# Convex hulls by group (optional)
for grp, color in color_map.items():
    pts = plot_df.loc[plot_df['cluster']==grp, ['total_income','total_spent_after_wages']].dropna()
    if len(pts) >= 3:
        try:
            hull = ConvexHull(pts.values)
            hull_pts = pts.values[hull.vertices]
            fig.add_trace(go.Scatter(
                x=np.r_[hull_pts[:,0], hull_pts[0,0]],
                y=np.r_[hull_pts[:,1], hull_pts[0,1]],
                mode='lines',
                line=dict(color=color, width=2, dash='dash'),
                name=f"{grp} Hull",
                showlegend=False,
                hoverinfo='skip'
            ))
        except Exception:
            pass

# Layout
fig.update_layout(
    title=f"Club Transfer Behavior ({start_year}–{last_year})",
    template='plotly_white',
    xaxis_title="Total Income (€)",
    yaxis_title="Total Spent + Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    legend=dict(orientation='h', yanchor='bottom', y=1.05, xanchor='left', x=0.25)
)
fig.update_xaxes(range=[0, x_max], tickformat=",.0f")
fig.update_yaxes(range=[0, y_max], tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "club_transfer_behavior_clustering_kmeans.png",
    width=1280, height=720, scale=2
)
```



5. Efficient Frontier

In this section, we will calculate the Efficient Frontier in football. This is the optimal point of money spent vs. percentage of wins.

5.1. Data Cleaning

In [34]:

```
# Check missing values in the dataset
missing_values = df_club_games.isnull().sum()

missing_values
```

Out[34]:

```
game_id                      0
club_id                     18
own_goals                   24
own_position             44934
own_manager_name          1656
opponent_id                 18
opponent_goals              24
opponent_position        44934
opponent_manager_name     1656
hosting                      0
is_win                       0
dtype: int64
```

In [35]:

```
# Create a new dataframe with the cleaned data
df_club_games_cleaned = df_club_games.dropna(
  subset=['club_id']
).copy()

# Show the dataset information that have been recently cleaned
print(f"Dataset Shape: {df_club_games_cleaned.shape}")
#print(f"\nColumn Data Types:\n{df_club_games_cleaned.dtypes}")
print(f"\nMissing Values:\n{df_club_games_cleaned.isnull().sum()}")
```

```
Dataset Shape: (148034, 11)

Missing Values:
game_id                      0
club_id                      0
own_goals                    6
own_position             44916
own_manager_name          1638
opponent_id                  0
opponent_goals               6
opponent_position        44916
opponent_manager_name     1638
hosting                      0
is_win                       0
dtype: int64
```

In [36]:

```
# Win rate by club_id
winrate_by_club = (
    df_club_games
      .groupby('club_id')
      .agg(
         games_played = ('game_id', 'count'),
         wins         = ('is_win', 'sum')
      )
)

winrate_by_club['win_rate'] = (winrate_by_club['wins'] / winrate_by_club['games_played']) * 100

winrate_by_club = winrate_by_club.sort_values('win_rate', ascending=False)
```

In [37]:

```
# Does a club_id have more than one different name?
clubid_to_name = (
    df_clubs.groupby('club_id')['name']
    .nunique()
    .reset_index(name='unique_names')
    .query('unique_names > 1')
)

print("Club IDs with more than one name associated:")
display(clubid_to_name)

# Does a name appear with more than one club_id?
name_to_clubid = (
    df_clubs.groupby('name')['club_id']
    .nunique()
    .reset_index(name='unique_ids')
    .query('unique_ids > 1')
)

print("Names of clubs with more than one club_id associated:")
display(name_to_clubid)
```

```
Club IDs with more than one name associated:
```

club_id	unique_names

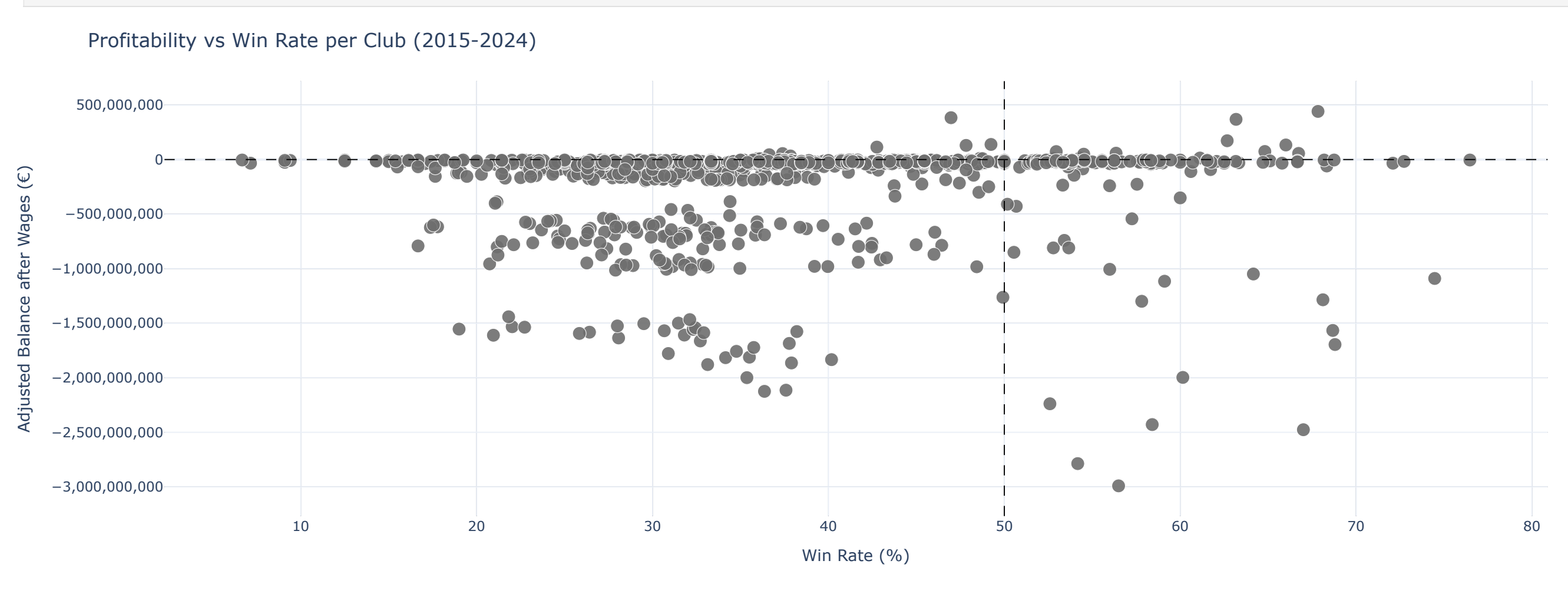
```
Names of clubs with more than one club_id associated:
```

name	unique_ids

5.2. Efficient Frontier Plotting

In [38]:

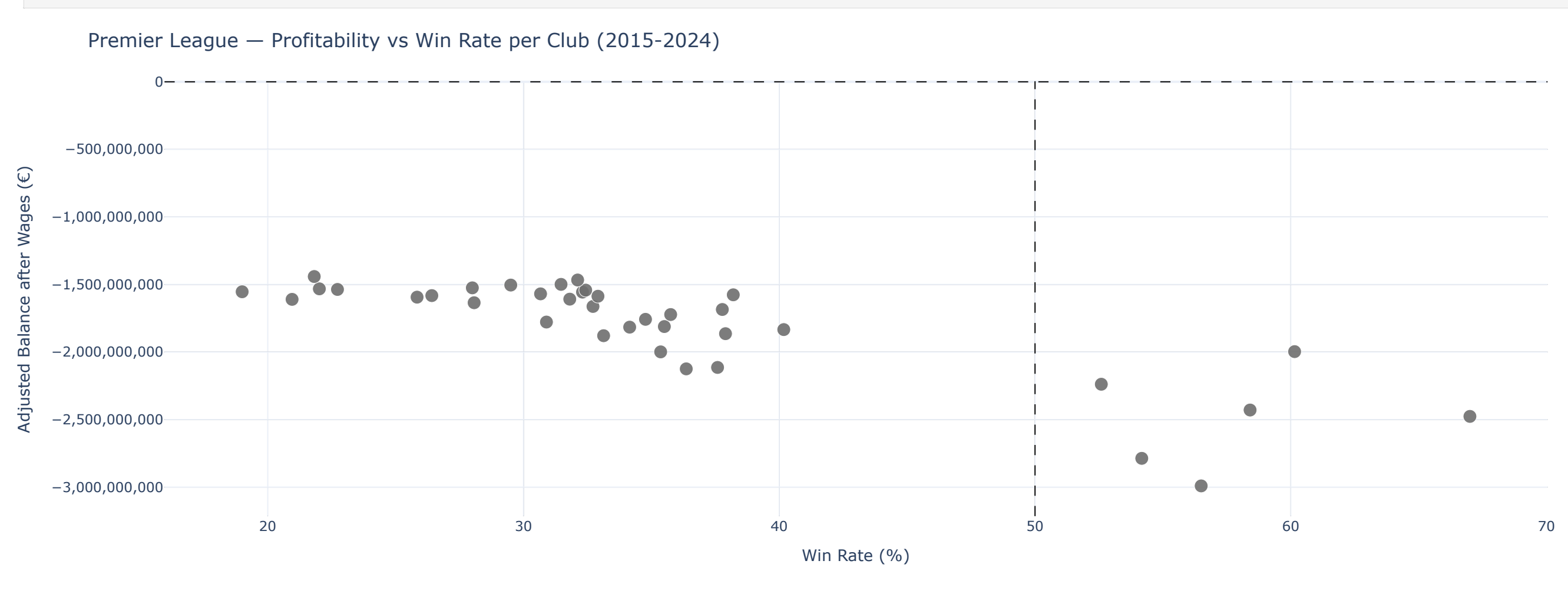
```
# Merge balances with winrates
club_perf = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['wins', 'games_played', 'win_rate']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name', 'domestic_competition_id']],
        on='club_id',
        how='left'
    )
)

# Clean NaNs and ensure numeric types
plot_df = (
    club_perf.dropna(subset=['win_rate', 'adj_balance_after_wages'])
             .query("games_played > 10")
             .copy()
)
plot_df['win_rate'] = pd.to_numeric(plot_df['win_rate'], errors='coerce')
plot_df['adj_balance_after_wages'] = pd.to_numeric(plot_df['adj_balance_after_wages'], errors='coerce')

# Scatter plot in gray, fixed size
fig = px.scatter(
    plot_df,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True,
        'domestic_competition_id': True
    },
    title="Profitability vs Win Rate per Club (2015–2024)"
)

# Style: all gray and same size
fig.update_traces(
    marker=dict(
        size=12,
        color='#6e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    ),
    selector=dict(mode='markers')
)

# Reference lines (optional): Y=0 (balance 0) and X=50% (win rate 50)
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout and formats
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)
fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "profitability_vs_winrate_gray.png",
    width=1280,
    height=720,
    scale=2
)
```



5.2.1. Efficient Frontier Plotting - Premier League

In [39]:

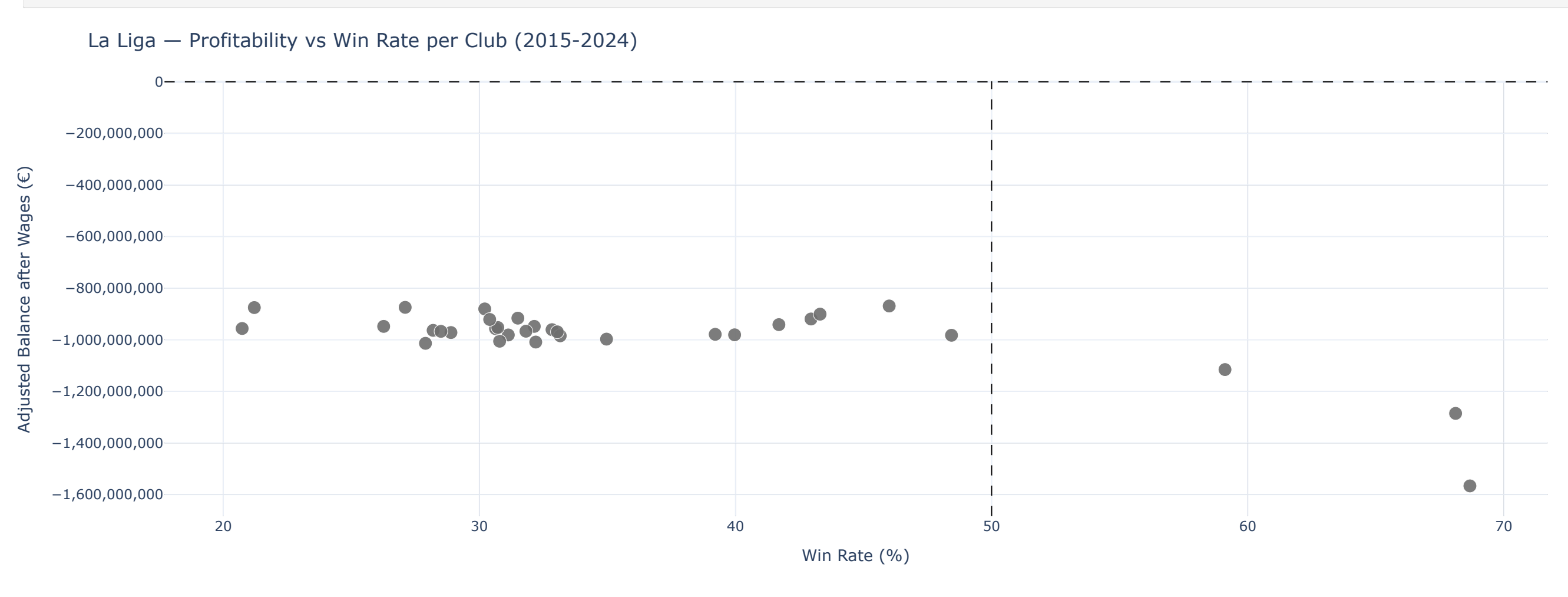
```
# Merge balances with winrates
club_perf = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['wins', 'games_played', 'win_rate']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name', 'domestic_competition_id']],
        on='club_id',
        how='left'
    )
)

# Clean NaNs, filter: >=10 partidos y solo Premier League (GB1)
plot_df = (
    club_perf.dropna(subset=['win_rate', 'adj_balance_after_wages'])
             .query("games_played > 10 and domestic_competition_id == 'GB1'")
             .copy()
)
plot_df['win_rate'] = pd.to_numeric(plot_df['win_rate'], errors='coerce')
plot_df['adj_balance_after_wages'] = pd.to_numeric(plot_df['adj_balance_after_wages'], errors='coerce')

# Scatter plot en gris, tamaño fijo
fig = px.scatter(
    plot_df,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True,
        'domestic_competition_id': True
    },
    title="Premier League — Profitability vs Win Rate per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#6e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    ),
    selector=dict(mode='markers')
)

# Líneas de referencia
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout y formatos
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)
fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "premier_profitability_vs_winrate_gray.png",
    width=1280,
    height=720,
    scale=2
)
```



5.2.2. Efficient Frontier Plotting - La Liga

In [40]:

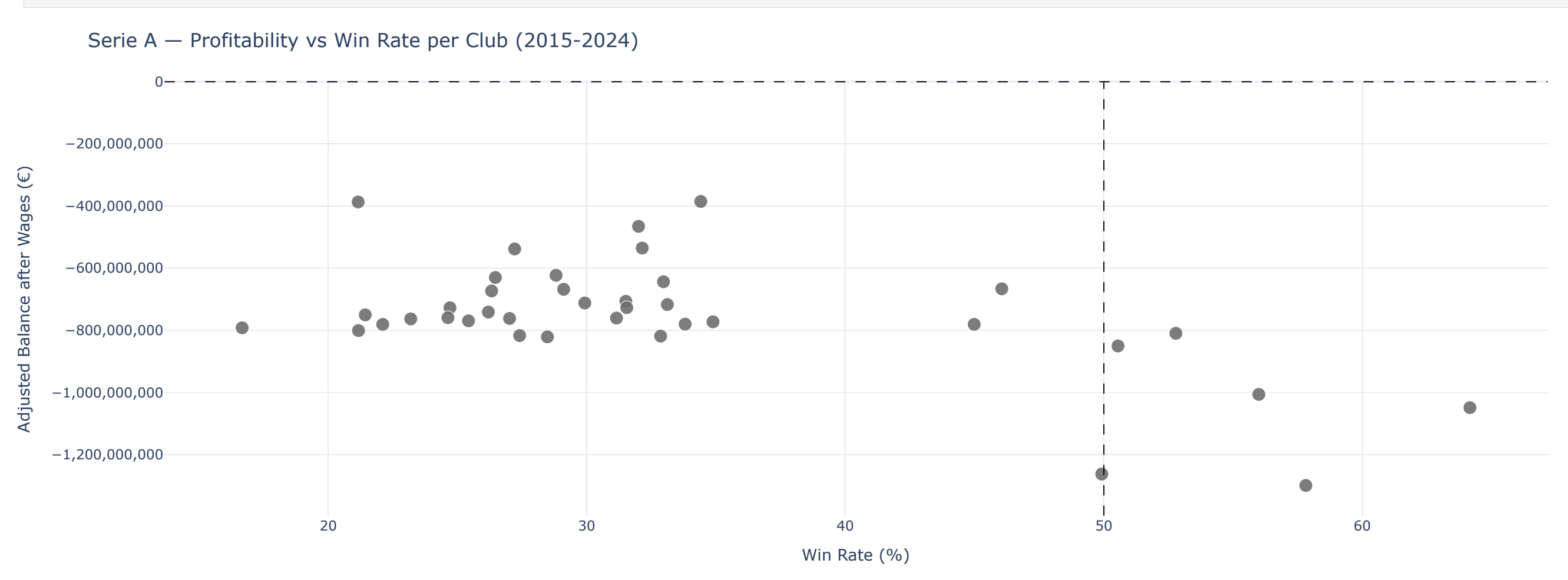
```
# Merge balances with winrates
club_perf = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['wins', 'games_played', 'win_rate']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name', 'domestic_competition_id']],
        on='club_id',
        how='left'
    )
)

# Clean NaNs, filter: >10 matches and only La Liga (ES1)
plot_df = (
    club_perf.dropna(subset=['win_rate', 'adj_balance_after_wages'])
             .query("games_played > 10 and domestic_competition_id == 'ES1'")
             .copy()
)
plot_df['win_rate'] = pd.to_numeric(plot_df['win_rate'], errors='coerce')
plot_df['adj_balance_after_wages'] = pd.to_numeric(plot_df['adj_balance_after_wages'], errors='coerce')

# Scatter plot in gray, fixed size
fig = px.scatter(
    plot_df,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True,
        'domestic_competition_id': True
    },
    title="La Liga — Profitability vs Win Rate per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#6e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    ),
    selector=dict(mode='markers')
)

# Reference lines
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout and formats
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)
fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "laliga_profitability_vs_winrate_gray.png",
    width=1280,
    height=720,
    scale=2
)
```



5.2.3. Efficient Frontier Plotting - Serie A

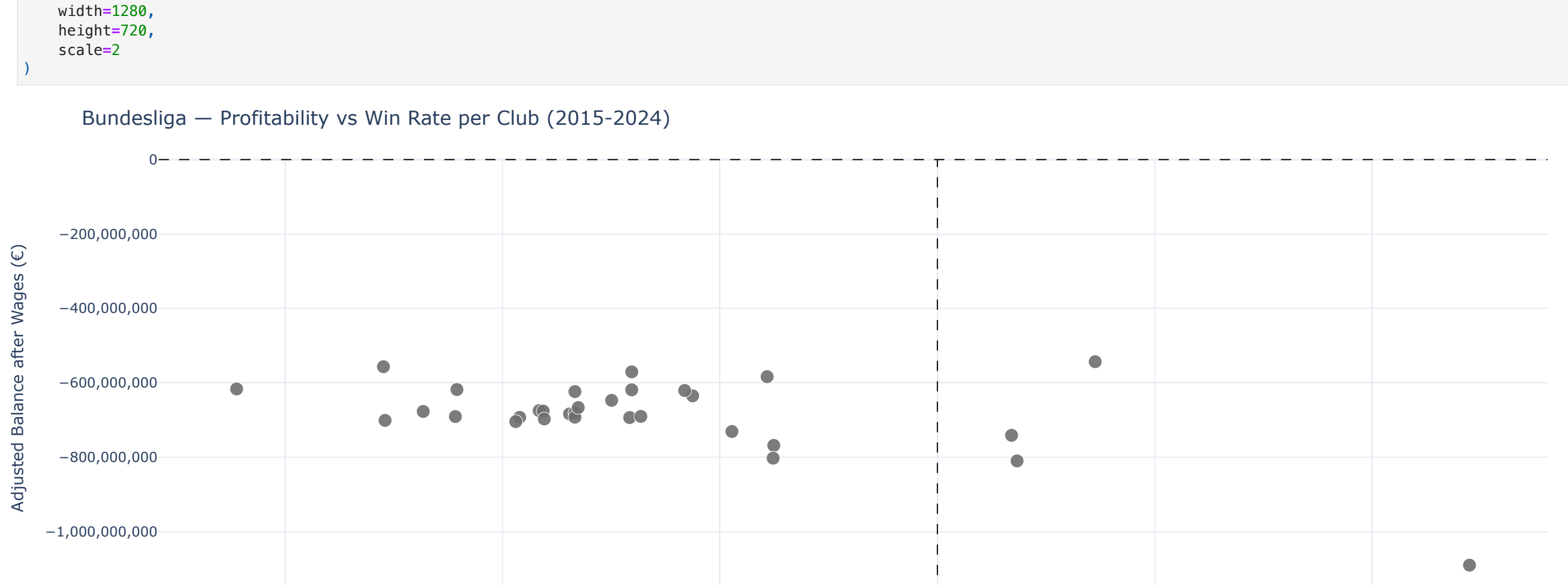
In [41]:

```
# Merge balances with winrates
club_perf = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['wins', 'games_played', 'win_rate']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name', 'domestic_competition_id']],
        on='club_id',
        how='left'
    )
)

# Clean NaNs, filter: >10 matches and only Serie A (IT1)
plot_df = (
    club_perf.dropna(subset=['win_rate', 'adj_balance_after_wages'])
             .query("games_played > 10 and domestic_competition_id == 'IT1'")
             .copy()
)
plot_df['win_rate'] = pd.to_numeric(plot_df['win_rate'], errors='coerce')
plot_df['adj_balance_after_wages'] = pd.to_numeric(plot_df['adj_balance_after_wages'], errors='coerce')

# Scatter plot in gray, fixed size
fig = px.scatter(
    plot_df,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True,
        'domestic_competition_id': True
    },
    title="Serie A — Profitability vs Win Rate per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#6e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    ),
    selector=dict(mode='markers')
)

# Reference lines
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout y formatos
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)
fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "serie_a_profitability_vs_winrate_gray.png",
    width=1280,
    height=720,
    scale=2
)
```



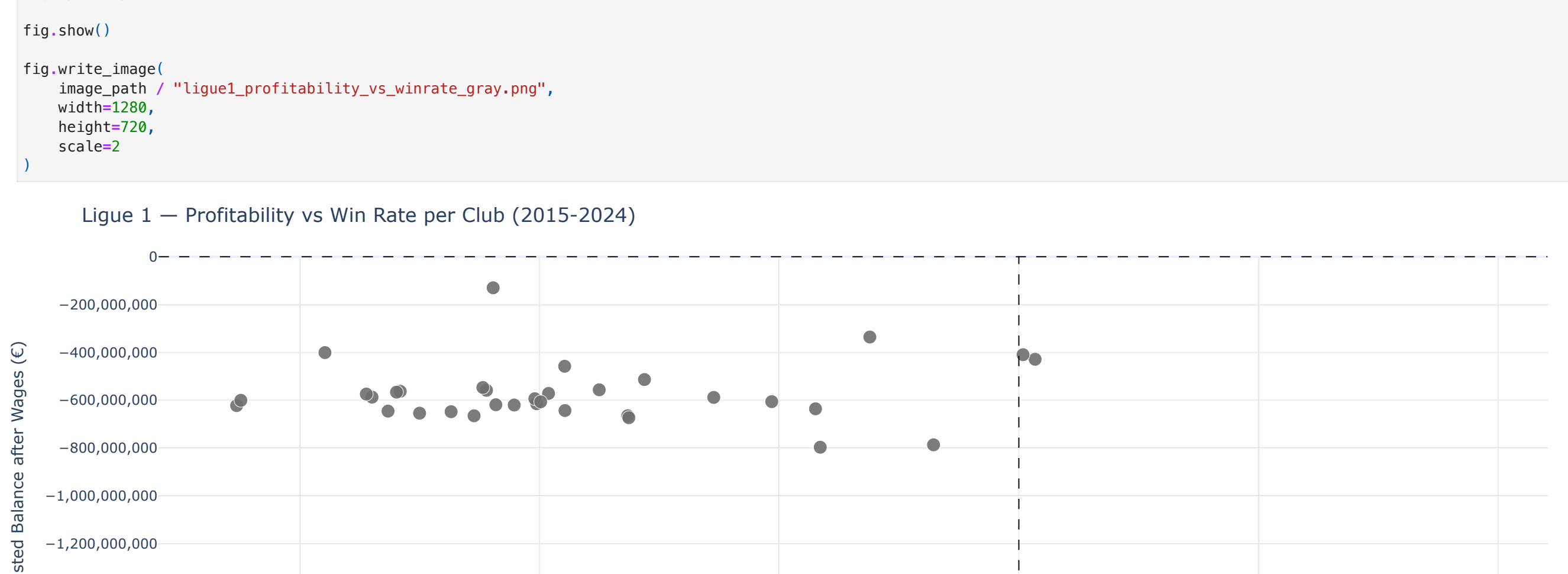
5.2.4. Efficient Frontier Plotting - Bundesliga

In [42]:
```
# Merge balances with winrates
club_perf = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['wins', 'games_played', 'win_rate']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name', 'domestic_competition_id']],
        on='club_id',
        how='left'
    )
)

# Clean NaNs, filter: >10 games and only Bundesliga (L1)
plot_df = (
    club_perf.dropna(subset=['win_rate', 'adj_balance_after_wages'])
             .query("games_played > 10 and domestic_competition_id == 'L1'")
             .copy()
)
plot_df['win_rate'] = pd.to_numeric(plot_df['win_rate'], errors='coerce')
plot_df['adj_balance_after_wages'] = pd.to_numeric(plot_df['adj_balance_after_wages'], errors='coerce')

# Scatter plot in gray, fixed size
fig = px.scatter(
    plot_df,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True,
        'domestic_competition_id': True
    },
    title="Bundesliga — Profitability vs Win Rate per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#6e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    ),
    selector=dict(mode='markers')
)

# Reference lines
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout and formats
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)
fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "bundesliga_profitability_vs_winrate_gray.png",
    width=1280,
    height=720,
    scale=2
)
```



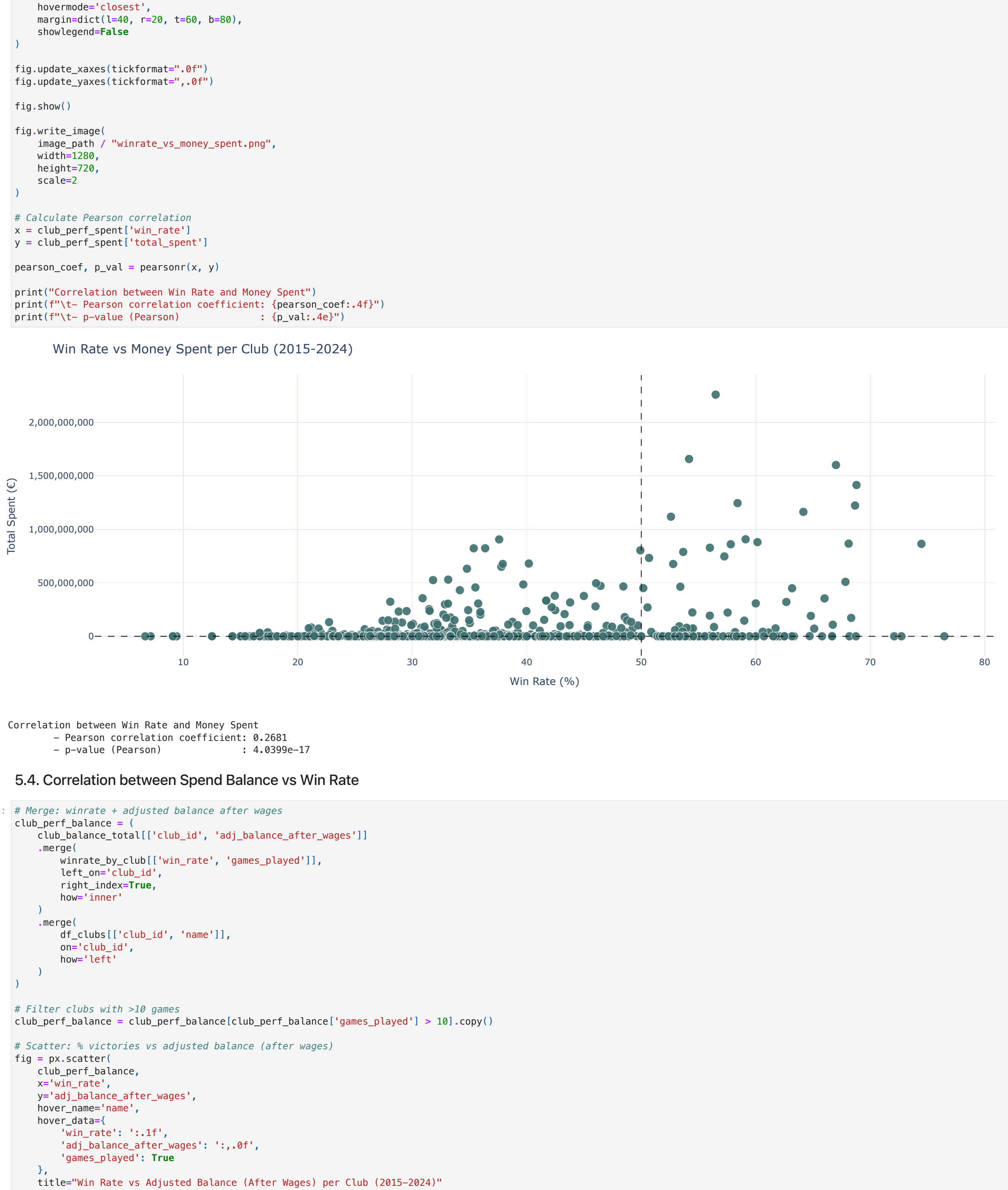
5.2.5. Efficient Frontier Plotting - Ligue 1

In [43]:
```
# Merge balances with winrates
club_perf = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['wins', 'games_played', 'win_rate']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name', 'domestic_competition_id']],
        on='club_id',
        how='left'
    )
)

# Clean NaNs, filter: >10 matches and only Ligue 1 (FR1)
plot_df = (
    club_perf.dropna(subset=['win_rate', 'adj_balance_after_wages'])
             .query("games_played > 10 and domestic_competition_id == 'FR1'")
             .copy()
)
plot_df['win_rate'] = pd.to_numeric(plot_df['win_rate'], errors='coerce')
plot_df['adj_balance_after_wages'] = pd.to_numeric(plot_df['adj_balance_after_wages'], errors='coerce')

# Scatter plot in gray, fixed size
fig = px.scatter(
    plot_df,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True,
        'domestic_competition_id': True
    },
    title="Ligue 1 — Profitability vs Win Rate per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#6e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    ),
    selector=dict(mode='markers')
)

# Reference lines
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout and formats
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)
fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "ligue1_profitability_vs_winrate_gray.png",
    width=1280,
    height=720,
    scale=2
)
```



5.3. Correlation between Money Spent vs Win Rate

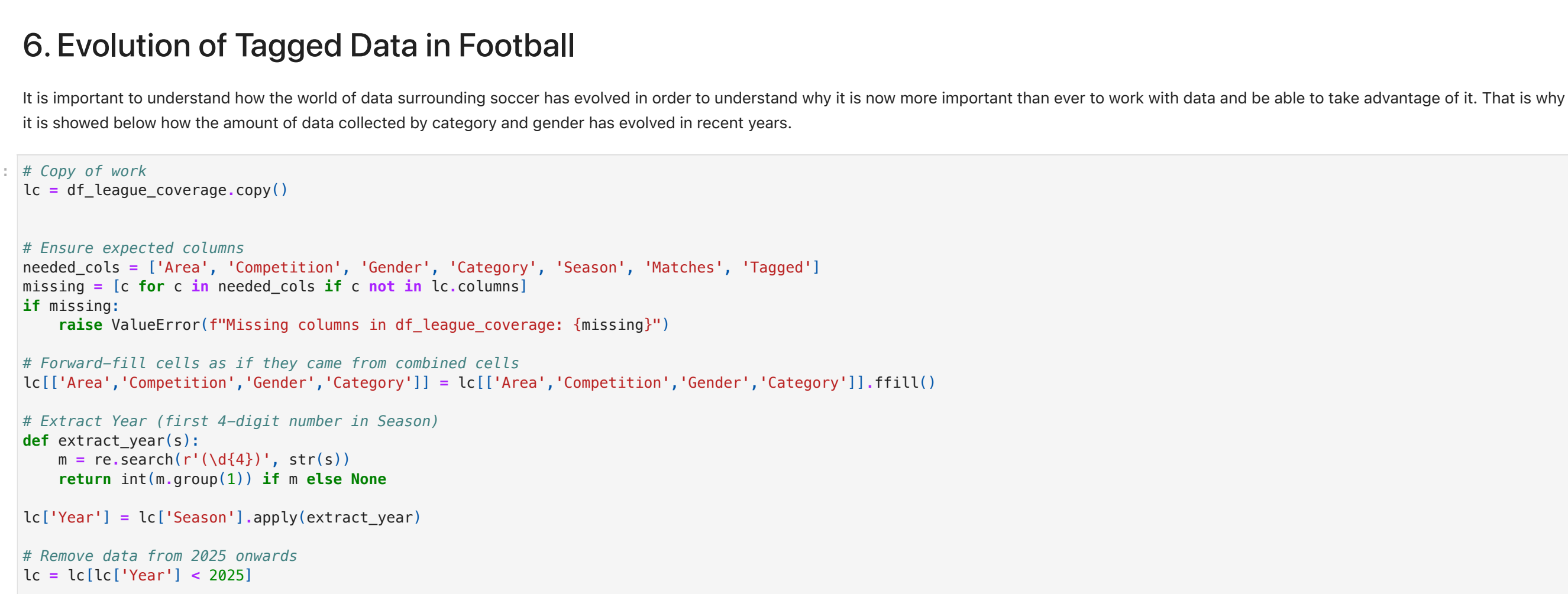
In [44]:
```
# Merge: winrate + spending
club_perf_spent = (
    club_balance_total[['club_id', 'total_spent']]
    .merge(
        winrate_by_club[['win_rate', 'games_played']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name']],
        on='club_id',
        how='left'
    )
)

# Filter clubs with games played (>0)
club_perf_spent = club_perf_spent[club_perf_spent['games_played'] > 10].copy()

# Scatter: % victories vs spending
fig = px.scatter(
    club_perf_spent,
    x='win_rate',
    y='total_spent',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'total_spent': ':,.0f',
        'games_played': True
    },
    title="Win Rate vs Money Spent per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#3e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    )
)

# Optional: reference lines
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Total Spent (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)

fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "winrate_vs_money_spent.png",
    width=1280,
    height=720,
    scale=2
)

# Calculate Pearson correlation
x = club_perf_spent['win_rate']
y = club_perf_spent['total_spent']

pearson_coef, p_val = pearsonr(x, y)

print("Correlation between Win Rate and Money Spent")
print(f"\t- Pearson correlation coefficient: {pearson_coef:.4f}")
print(f"\t- p-value (Pearson)              : {p_val:.4e}")
```



```
Correlation between Win Rate and Money Spent
        - Pearson correlation coefficient: 0.2681
        - p-value (Pearson)              : 4.0359e-17
```

5.4. Correlation between Spend Balance vs Win Rate

In [45]:
```
# Merge: winrate + adjusted balance after wages
club_perf_balance = (
    club_balance_total[['club_id', 'adj_balance_after_wages']]
    .merge(
        winrate_by_club[['win_rate', 'games_played']],
        left_on='club_id',
        right_index=True,
        how='inner'
    )
    .merge(
        df_clubs[['club_id', 'name']],
        on='club_id',
        how='left'
    )
)

# Filter clubs with >10 games
club_perf_balance = club_perf_balance[club_perf_balance['games_played'] > 10].copy()

# Scatter: % victories vs adjusted balance (after wages)
fig = px.scatter(
    club_perf_balance,
    x='win_rate',
    y='adj_balance_after_wages',
    hover_name='name',
    hover_data={
        'win_rate': ':.1f',
        'adj_balance_after_wages': ':,.0f',
        'games_played': True
    },
    title="Win Rate vs Adjusted Balance (After Wages) per Club (2015–2024)"
)

fig.update_traces(
    marker=dict(
        size=12,
        color='#3e6e6e',
        opacity=0.9,
        line=dict(width=0.6, color='white')
    )
)

# Reference lines
fig.add_hline(y=0, line_color='black', line_width=1, line_dash='dash')
fig.add_vline(x=50, line_color='black', line_width=1, line_dash='dash')

# Layout
fig.update_layout(
    template='plotly_white',
    xaxis_title="Win Rate (%)",
    yaxis_title="Adjusted Balance after Wages (€)",
    hovermode='closest',
    margin=dict(l=40, r=20, t=60, b=80),
    showlegend=False
)

fig.update_xaxes(tickformat=".0f")
fig.update_yaxes(tickformat=",.0f")

fig.show()

fig.write_image(
    image_path / "winrate_vs_adj_balance_after_wages.png",
    width=1280,
    height=720,
    scale=2
)

# Calculate Pearson correlation
x = club_perf_balance['win_rate']
y = club_perf_balance['adj_balance_after_wages']

pearson_coef, p_val = pearsonr(x, y)

print("Correlation between Win Rate and Adjusted Balance (After Wages)")
print(f"\t- Pearson correlation coefficient: {pearson_coef:.4f}")
print(f"\t- p-value (Pearson)              : {p_val:.4e}")
```

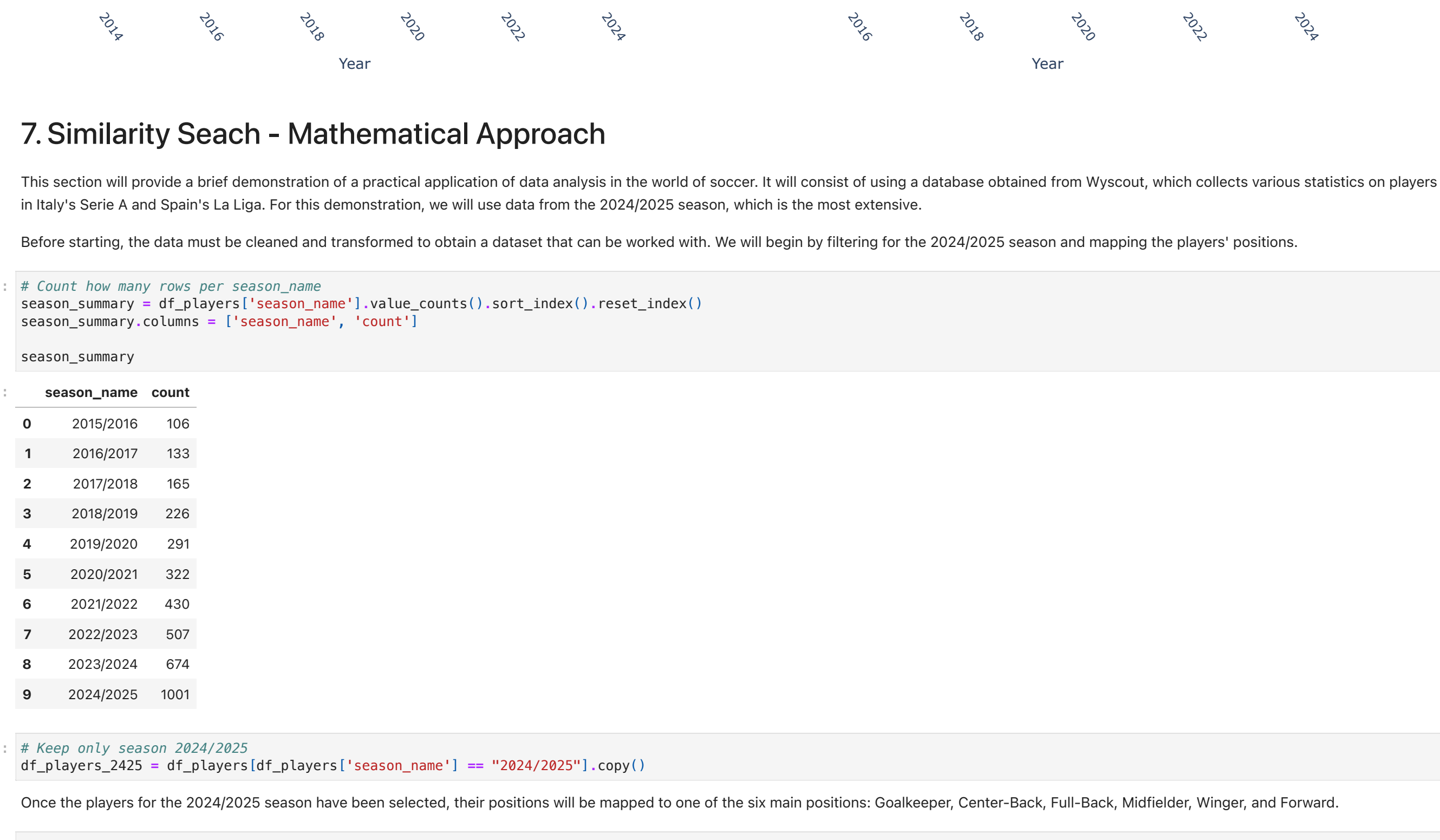


```
Correlation between Win Rate and Adjusted Balance (After Wages)
        - Pearson correlation coefficient: 0.0534
        - p-value (Pearson)              : 9.9733e-02
```

6. Evolution of Tagged Data in Football

It is important to understand how the world of data surrounding soccer has evolved in order to understand why it is now more important than ever to work with data and be able to take advantage of it. That is why it is showed below how the amount of data collected by category and gender has evolved in recent years.

In [46]:
```
# Copy of work
lc = df_league_coverage.copy()


# Ensure expected columns
needed_cols = ['Area', 'Competition', 'Gender', 'Category', 'Season', 'Matches', 'Tagged']
missing = [c for c in needed_cols if c not in lc.columns]
if missing:
    raise ValueError(f"Missing columns in df_league_coverage: {missing}")

# Forward-fill cells as if they came from combined cells
lc[['Area','Competition','Gender','Category']] = lc[['Area','Competition','Gender','Category']].ffill()

# Extract Year (first 4-digit number in Season)
def extract_year(s):
    m = re.search(r'(\d{4})', str(s))
    return int(m.group(1)) if m else None

lc['Year'] = lc['Season'].apply(extract_year)

# Remove data from 2025 onwards
lc = lc[lc['Year'] < 2025]

# Normalize percentage
lc['PercentTagged'] = (
    lc['Tagged'].astype(str)
    .str.replace('%', '', regex=False)
    .str.strip()
    .replace({'': None})
    .astype(float)
)

# Ensure numeric
lc['Matches'] = pd.to_numeric(lc['Matches'], errors='coerce')

# Filter valid
lc = lc.dropna(subset=['Gender','Category','Year','PercentTagged'])

# Function that creates a fig style “transfer count”
def make_league_fig(df_gc: pd.DataFrame, gender: str, category: str):
    # Add by year (average of % if there are several entries for the same year)
    counts = (
        df_gc.groupby('Year', as_index=False)
             .agg(pct=('PercentTagged','mean'),
                  n=('PercentTagged','size'))
             .sort_values('Year')
             .reset_index(drop=True)
    )
    # Rolling 3 centered
    counts['roll3'] = counts['pct'].rolling(3, center=True, min_periods=2).mean()

    # Bars
    fig = px.bar(
        counts,
        x='Year',
        y='pct',
        color='pct',
        text=counts['pct'].round(0),
        title=f'% Competitions Tagged by Year – {gender} / {category}',
        hover_data={'Year': True, 'pct': ':.1f', 'n': True}
    )
    fig.update_traces(textposition='outside', cliponaxis=False)

    # Trend line
    fig.add_trace(go.Scatter(
        x=counts['Year'],
        y=counts['roll3'],
        mode='lines+markers',
        name='3Y Moving Avg',
        hovertemplate='Year: %{x}<br>3Y Moving Avg: %{y:.1f}%<extra></extra>'
    ))

    # Style
    fig.update_layout(
        legend=dict(orientation='v', yanchor='top', y=1.05, xanchor='left', x=0),
        template='plotly_dark',
        xaxis_title='Year',
        yaxis_title='% Competitions Tagged',
        hovermode='x unified',
        margin=dict(l=40, r=20, t=60, b=80),
        xaxis=dict(
            tickmode='array',
            tickvals=counts['Year'],
            ticktext=counts['Year'].astype(str),
            tickangle=55,
            rangeslider=dict(visible=False),
            rangeselector=dict(visible=False)
        ),
        coloraxis_colorbar=dict(title='% Tagged')
    )
    fig.update_yaxes(ticksuffix='%')

    return fig, counts

# Generate a graph by combination Gender–Category
# If you want “exactly 4”, this order fixes the iteration.
target_pairs = [('Male','Senior'), ('Male','Youth'), ('Female','Senior'), ('Female','Youth')]

figs_info = []  # (gender, category, fig, table)
available_pairs = sorted(lc[['Gender','Category']].drop_duplicates().itertuples(index=False), key=lambda x: (x[0], x[1]))

for gender, category in target_pairs:
    sub = lc[(lc['Gender']==gender) & (lc['Category']==category)]
    if len(sub)==0:
        print(f"Warning: no data for {gender}/{category}, it is omitted.")
        continue
    fig, table = make_league_fig(sub, gender, category)
    figs_info.append((gender, category, fig, table))


# Show individually (in notebook)
# for _, _, fig, _ in figs_info:
#     fig.show()

# Dashboard 2×2 with make_subplots if there are up to 4 figs
if 1 <= len(figs_info) <= 4:
    rows = 2
    cols = 2
    dashboard = make_subplots(
        rows=rows, cols=cols,
        subplot_titles=[f"{g} / {c}" for g, c, _, _ in figs_info]
    )
    r = c = 1
    for gender, category, fig, table in figs_info:
        bar_tr = [tr for tr in fig.data if isinstance(tr, go.Bar)][0]
        line_tr = [tr for tr in fig.data if isinstance(tr, go.Scatter)][0]
        dashboard.add_trace(bar_tr, row=r, col=c)
        dashboard.add_trace(line_tr, row=r, col=c)

        # Local axes and style
        dashboard.update_xaxes(title_text="Year", row=r, col=c, tickangle=55)
        dashboard.update_yaxes(title_text="% Tagged", ticksuffix="%", row=r, col=c)

        c += 1
        if c > cols:
            c = 1
            r += 1

    dashboard.update_layout(
        template='plotly_white',
        title_text='League Coverage – % Tagged by Year',
        showlegend=False,
        margin=dict(l=40, r=20, t=60, b=60),
        hovermode='x unified',
        height=800
    )

    dashboard.show()

    dashboard.write_image(
        image_path / "tagged_competitions.png",
        width=1280,
        height=720,
        scale=2
    )
```



7. Similarity Seach - Mathematical Approach

This section will provide a brief demonstration of a practical application of data analysis in the world of soccer. It will consist of using a database obtained from Wyscout, which collects various statistics on players in Italy's Serie A and Spain's La Liga. For this demonstration, we will use data from the 2024/2025 season, which is the most extensive.

Before starting, the data must be cleaned and transformed to obtain a dataset that can be worked with. We will begin by filtering for the 2024/2025 season and mapping the players' positions.

In [47]:
```
# Count how many rows per season_name
season_summary = df_players['season_name'].value_counts().sort_index().reset_index()
season_summary.columns = ['season_name', 'count']

season_summary
```

Out[47]:

	season_name	count
0	2015/2016	106
1	2016/2017	133
2	2017/2018	165
3	2018/2019	226
4	2019/2020	291
5	2020/2021	322
6	2021/2022	430
7	2022/2023	507
8	2023/2024	674
9	2024/2025	1001

In [48]:
```
# Keep only season 2024/2025
df_players_2425 = df_players[df_players['season_name'] == "2024/2025"].copy()
```

Once the players for the 2024/2025 season have been selected, their positions will be mapped to one of the six main positions: Goalkeeper, Center-Back, Full-Back, Midfielder, Winger, and Forward.

In [49]:
```
# Get the summary of the positions of the players
def parse_positions(x):
    try:
        return ast.literal_eval(x) if isinstance(x, str) else x
    except Exception:
        return None

df_players_2425['positions'] = df_players_2425['positions'].apply(parse_positions)

# Extract code, name, percent
def extract_code(entry):
    if isinstance(entry, dict) and 'position' in entry:
        return entry['position'].get('code')
    return None

def extract_name(entry):
    if isinstance(entry, dict) and 'position' in entry:
        return entry['position'].get('name')
    return None

def extract_percent(entry):
    if isinstance(entry, dict):
        return entry.get('percent')
    return None

df_players_2425['pos_code']     = df_players_2425['positions'].apply(extract_code)
df_players_2425['pos_name']     = df_players_2425['positions'].apply(extract_name)
df_players_2425['pos_percent']  = df_players_2425['positions'].apply(extract_percent)

# Quick overview: all unique codes and their counts
codes_summary = df_players_2425['pos_code'].value_counts().reset_index()
codes_summary.columns = ['pos_code', 'count']
codes_summary
```

Out[49]:

	pos_code	count
0	cf	147
1	gk	77
2	rb	69
3	rcb	60
4	lcb	57
5	lb	57
6	amf	51
7	rcmf	44
8	lcmf	43
9	lw	36
10	rw	35
11	ldmf	33
12	lamf	33
13	ramf	32
14	rdmf	30
15	rwb	28
16	rcb3	25
17	lcb3	24
18	dmf	22
19	lwb	21
20	rcmf3	19
21	cb	19
22	lcmf3	18
23	lwf	8
24	rwf	6
25	rb5	4
26	lb5	3

In [50]:
```
# Define mapping from pos_code -> final_position
code_to_final = {
    # Goalkeeper
    "gk": "Goalkeeper",

    # Centre-Back (includes variants of 3/5 line)
    "cb": "Centre-Back", "rcb": "Centre-Back", "lcb": "Centre-Back",
    "rcb3": "Centre-Back", "lcb3": "Centre-Back",

    # Full-Back (laterales and carrileros)
    "rb": "Full-Back", "lb": "Full-Back",
    "rwb": "Full-Back", "lwb": "Full-Back",
    "rb5": "Full-Back", "lb5": "Full-Back",

    # Midfielders
    "cmf": "Midfielder", "rcmf": "Midfielder", "lcmf": "Midfielder",
    "rcmf3": "Midfielder", "lcmf3": "Midfielder",
    "dmf": "Midfielder", "ldmf": "Midfielder", "rdmf": "Midfielder",
    "amf": "Midfielder", "lamf": "Midfielder", "ramf": "Midfielder",

    # Wingers
    "lw": "Winger", "rw": "Winger",
    "lwf": "Winger", "rwf": "Winger",

    # Forwards
    "cf": "Forward", "st": "Forward", "ss": "Forward"
}

# Add column to dataframe
df_players_2425['final_position'] = df_players_2425['pos_code'].map(code_to_final).fillna("Midfielder")

# Quick check: counts per final_position
df_players_2425['final_position'].value_counts()
```

Out[50]:
```
final_position
Midfielder     325
Centre-Back    185
Full-Back      182
Forward        147
Winger          85
Goalkeeper      77
Name: count, dtype: int64
```

When players are analyzed within the same role (e.g., goalkeepers or midfielders), it is unlikely that all of them share exactly the same profile. Instead, there are sub-types of players: a defensive midfielder versus a creative playmaker, or a shot-stopping goalkeeper versus a sweeper-keeper.

Clustering helps us to discover these hidden subgroups automatically. However, the number of clusters (k) is not obvious:

- If choose k too low, it oversimplifies and lose meaningful distinctions.
- If choose k too high, it fragments the data and create artificial clusters.

Therefore, selecting the optimal k is a crucial step. Techniques such as the Elbow Method or the Silhouette Score provide statistical guidance on how many clusters best represent the diversity of players in each role. Once the clusters are well-defined, which variables vary most across them can be measured, and those become the key features to characterize players in that role.

This ensures that our similarity search later on is based on the most discriminative and role-specific attributes, rather than arbitrary or redundant statistics.

In [51]:
```
def optimal_k_for_roles(df, roles=None, k_range=range(2,11)):
    results = {}

    if roles is None:
        roles = df['final_position'].unique()

    for role in roles:
        role_df = df[df['final_position'] == role]

        # keep only numeric columns
        numeric_cols = role_df.select_dtypes(include='number').columns
        X = role_df[numeric_cols].dropna(axis=1, how="any")

        if X.shape[0] < 10 or X.shape[1] < 2:
            print(f"Not enough data for {role}, skipping.")
            continue

        # Standardize
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        best_k, best_score = None, -1
        for k in k_range:
            try:
                kmeans = KMeans(n_clusters=k, random_state=42, n_init="auto").fit(X_scaled)
                score = silhouette_score(X_scaled, kmeans.labels_)
                if score > best_score:
                    best_k, best_score = k, score
            except Exception:
                continue

        results[role] = {"best_k": best_k, "silhouette": best_score}

    return results

# Run for 2024/2025 players
k_results = optimal_k_for_roles(df_players_2425, k_range=range(2,11))

pd.DataFrame(k_results).T
```

Out[51]:

	best_k	silhouette
Midfielder	3.0	0.150673
Full-Back	2.0	0.182108
Centre-Back	3.0	0.178888
Goalkeeper	2.0	0.150524
Forward	3.0	0.222320
Winger	3.0	0.180304

Once the optimal number of clusters has been determined for each position, the next step is to analyze which features best distinguish player sub–types.

By comparing the centroids of the clusters, we can measure the variance of each attribute across the groups. Features with higher variance are those that separate clusters most strongly, and therefore represent the key dimensions of playing style within that role.

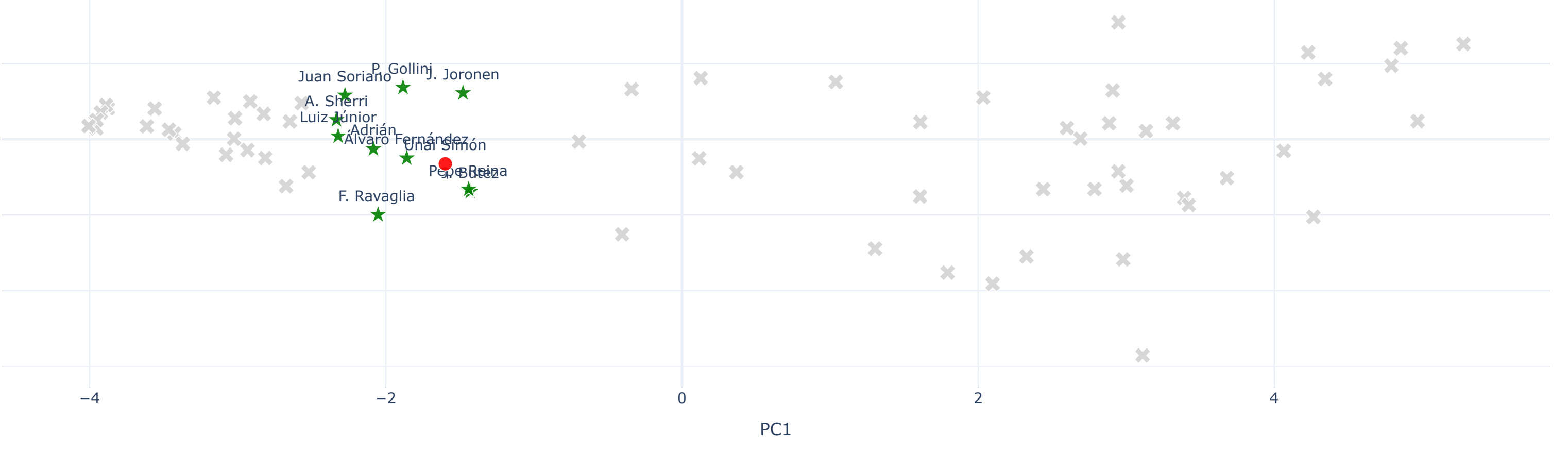
This process allows us to go beyond intuition (e.g., "forwards score goals") and statistically identify which variables are the most informative to characterize players in each role.

In [52]:
```
def important_features_per_role(df, k_results, top_n=5):
    role_features = {}

    for role, row in k_results.iterrows():
        k = int(row['best_k'])
        role_df = df[df['final_position'] == role]

        numeric_cols = role_df.select_dtypes(include='number').columns
        X = role_df[numeric_cols].dropna(axis=1, how="any")

        if X.shape[1] < 2:
            continue

        # Standardize
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # Run KMeans with chosen k
        kmeans = KMeans(n_clusters=k, random_state=42, n_init="auto").fit(X_scaled)

        # Compute variance across cluster centroids
        cluster_centers = pd.DataFrame(kmeans.cluster_centers_, columns=X.columns)
        variance = cluster_centers.var().sort_values(ascending=False)

        top_features = list(variance.head(top_n).index)
        role_features[role] = top_features

    return role_features

# Convert your k_results dict to DataFrame if not already
k_results_df = pd.DataFrame({
    "best_k": {
        "Midfielder": 3.0,
        "Full-Back": 2.0,
        "Centre-Back": 3.0,
        "Goalkeeper": 2.0,
        "Forward": 3.0,
        "Winger": 3.0,
    },
    "silhouette": {
        "Midfielder": 0.150673,
        "Full-Back": 0.182108,
        "Centre-Back": 0.178888,
        "Goalkeeper": 0.150524,
        "Forward": 0.222320,
        "Winger": 0.180304,
    }
})

# Extract important features
important_features = important_features_per_role(df_players_2425, k_results_df, top_n=10)
```

Given a player, we look for their closest neighbors (with the important features of their position).

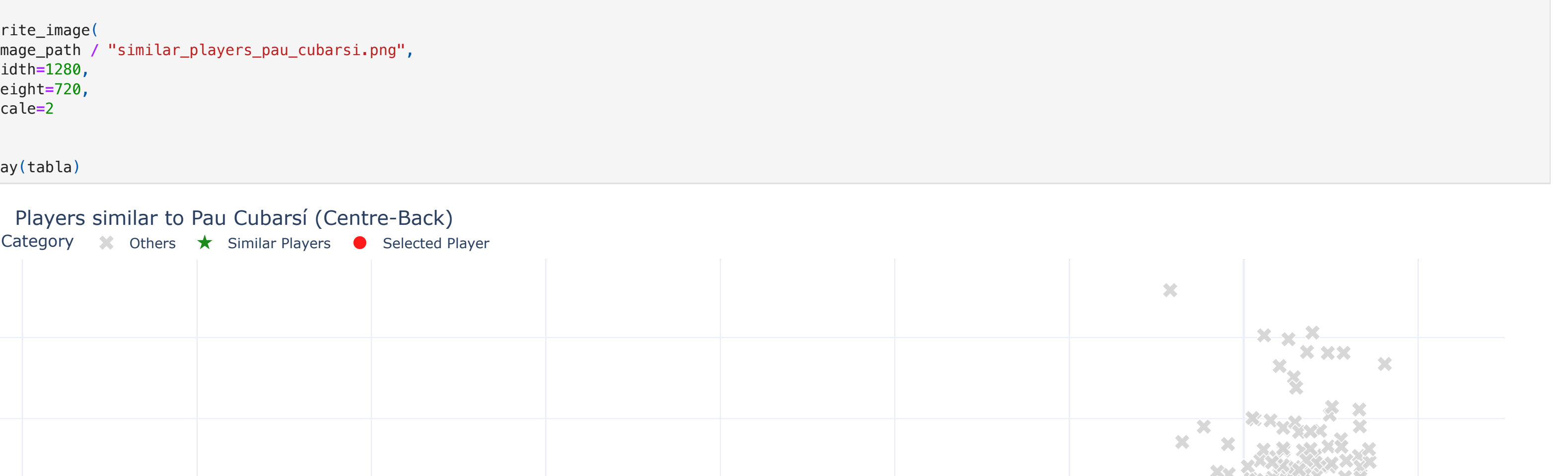
These features are reduced to 2D with PCA. They are drawn on a scatter plot: the selected player in red, similar players in green, and the rest in gray, with names as labels.

In [53]:
```
def plot_player_similarity(df, player_name, features_by_role, n_neighbors=10):
    # Player selection and features by role
    myplayer = df[df['player_name'] == player_name]
    if myplayer.empty:
        raise ValueError(f"Player {player_name} not found")

    role = myplayer.iloc[0]['final_position']
    features = features_by_role.get(role, [])
    if not features:
        raise ValueError(f"No features found for role {role}")

    # Filter by same role and align with non-null features
    same_role_players = df[df['final_position'] == role].copy()
    X = same_role_players[features].dropna()
    same_role_players = same_role_players.loc[X.index]

    # Ensure the player is present after dropna()
    idx = myplayer.index[0]
    if idx not in same_role_players.index:
        raise ValueError(f"Selected player '{player_name}' has missing values in required features: {features}")

    # Scaling
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Nearest neighbors
    knn = NearestNeighbors(n_neighbors=min(n_neighbors+1, len(same_role_players)))
    knn.fit(X_scaled)
    rel_index = same_role_players.index.get_loc(idx)
    distances, indices = knn.kneighbors(X_scaled[rel_index].reshape(1, -1))
    neighbor_positions = indices[0][1:]
    neighbor_distances = distances[0][1:]

    # PCA reduction to 2D (for plotting)
    pca = PCA(n_components=2)
    coords_2d = pca.fit_transform(X_scaled)
    same_role_players = same_role_players.copy()
    same_role_players['PC1'] = coords_2d[:, 0]
    same_role_players['PC2'] = coords_2d[:, 1]

    # Categories and visible labels
    same_role_players['Category'] = "Others"
    same_role_players.iloc[rel_index, same_role_players.columns.get_loc('Category')] = "Selected Player"
    same_role_players.iloc[neighbor_positions, same_role_players.columns.get_loc('Category')] = "Similar Players"

    # Label: only non-empty for selected/similar players
    same_role_players['label'] = np.where(
        same_role_players['Category'] != "Others",
        same_role_players['player_name'],
        ""
    )

    # Scatter Plot with different symbols by category
    color_map = {
        "Selected Player": "red",
        "Similar Players": "green",
        "Others": "lightgrey"
    }
    symbol_map = {
        "Selected Player": "circle",
        "Similar Players": "star",
        "Others": "x"
    }

    fig = px.scatter(
        same_role_players,
        x="PC1",
        y="PC2",
        color="Category",
        symbol="Category",
        text="label",
        color_discrete_map=color_map,
        symbol_map=symbol_map,
        category_orders={"Category": ["Others", "Similar Players", "Selected Player"]},
        title=f"Players similar to {player_name} ({role})"
    )

    fig.update_traces(
        marker=dict(size=12, opacity=0.9, line=dict(width=0.8, color='white')),
        textposition='top center'
    )
    fig.update_layout(
        template='plotly_white',
        xaxis_title="PC1",
        yaxis_title="PC2",
        hovermode='closest',
        margin=dict(l=40, r=20, t=60, b=80),
        legend=dict(orientation='h', yanchor='bottom', y=1.0, xanchor='left', x=0)
    )

    # Table: selected player + neighbors ordered by proximity
    selected_row = same_role_players.iloc[[rel_index]]
    neighbors_df  = same_role_players.iloc[neighbor_positions].copy()

    # Add Distance column (0 for selected, then the real ones)
    selected_row = selected_row.copy()
    selected_row['Distance'] = 0.0
    neighbors_df['Distance'] = neighbor_distances

    # Order neighbors by distance (just in case)
    neighbors_df = neighbors_df.sort_values('Distance', ascending=True)

    # Build final table with columns: name + features + distance
    cols_for_table = ['player_name'] + features + ['Distance']
    # Take the values of features from X
    selected_values = pd.concat([
        pd.DataFrame({
            'player_name': [selected_row.iloc[0]['player_name']],
            **{f: [X.loc[selected_row.index[0], f]] for f in features},
            'Distance': [0.0]
        })
    ], ignore_index=True)

    neighbor_values_list = []
    for i, idx_abs in enumerate(neighbors_df.index):
        row_dict = {'player_name': same_role_players.loc[idx_abs, 'player_name']}
        for f in features:
            row_dict[f] = X.loc[idx_abs, f]
        row_dict['Distance'] = neighbors_df.loc[idx_abs, 'Distance']
        neighbor_values_list.append(row_dict)
    neighbors_values = pd.DataFrame(neighbor_values_list)

    table_df = pd.concat([selected_values, neighbors_values], ignore_index=True)
    # Limit to n_neighbors (if there are less available, it's already controlled)
    if len(table_df) > (n_neighbors + 1):
        table_df = table_df.iloc[:n_neighbors + 1]

    return fig, table_df

fig, tabla = plot_player_similarity(df_players_2425, "Unai Simón", important_features, n_neighbors=10)
fig.show()

fig.write_image(
    image_path / "similar_players_unai_simon.png",
    width=1280,
    height=720,
    scale=2
)

display(tabla)
```



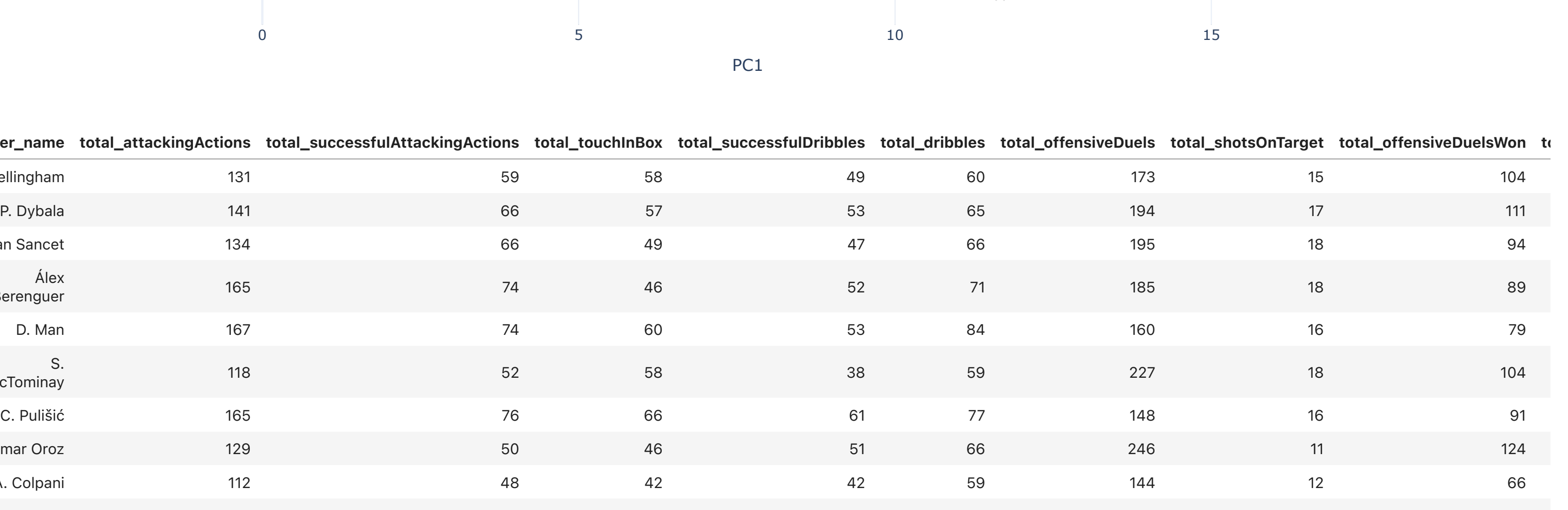
	player_name	total_matchesInStart	total_minutesOnField	total_minutesTagged	total_matches	total_gkShotsAgainst	total_gkSaves	total_recoveries	total_forwardPasses	total_xgSave	total_success
0	Unai Simón	9	869	869	9	28	20	33	109	10.04	
1	Álvaro Fernández	7	725	725	8	30	18	30	130	8.34	
2	Adrián	7	688	688	7	24	14	30	88	8.34	
3	Pepe Reina	10	980	980	10	37	25	25	140	7.71	
4	J. Joronen	7	764	764	8	42	27	36	112	12.97	
5	F. Ravaglia	8	774	774	8	17	8	35	116	6.35	
6	P. Gollini	7	705	705	7	34	19	21	80	13.77	
7	J. Butez	9	870	870	9	33	20	37	183	11.33	
8	Luiz Júnior	6	619	619	7	26	19	26	60	7.24	
9	A. Sherri	6	583	583	6	25	16	19	72	6.54	
10	Juan Soriano	6	595	595	6	27	20	15	61	7.80	

In [54]:
```
fig, tabla = plot_player_similarity(df_players_2425, "Pau Cubarsí", important_features, n_neighbors=10)
fig.show()

fig.write_image(
    image_path / "similar_players_pau_cubarsi.png",
    width=1280,
    height=720,
    scale=2
)

display(tabla)
```



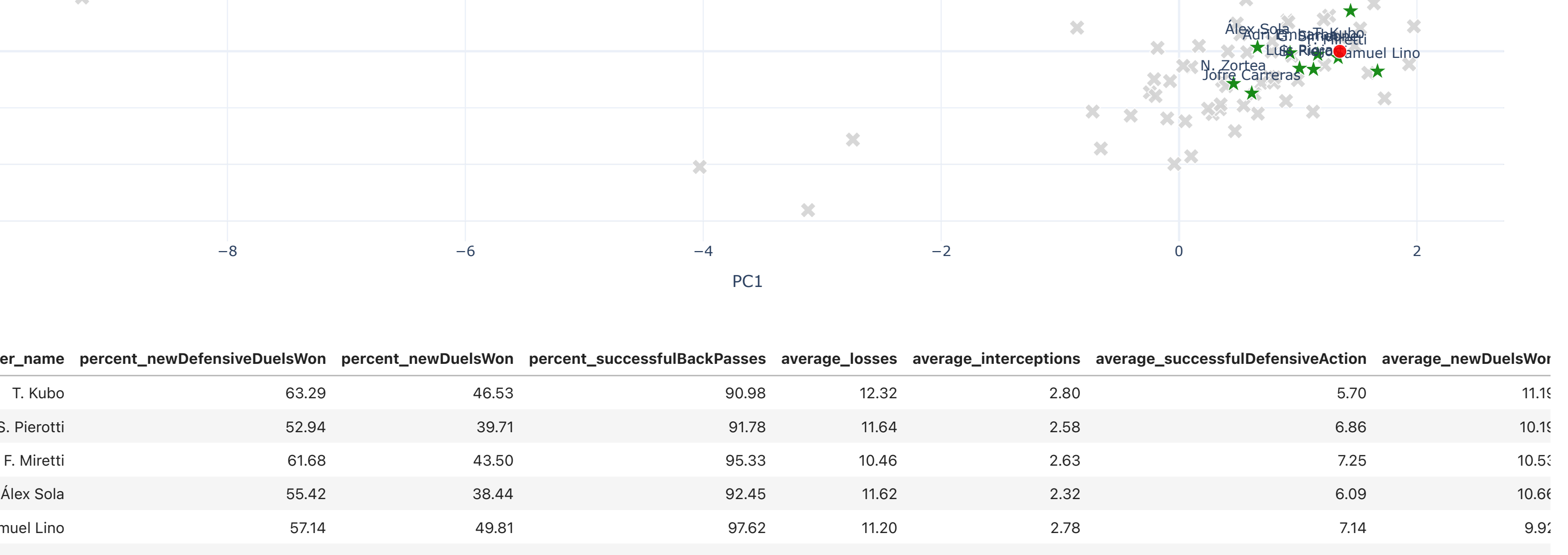
	player_name	percent_successfulLateralPasses	percent_successfulVerticalPasses	percent_successfulPasses	percent_successfulForwardPasses	percent_successfulBackPasses	percent_newDuelsWon
0	Pau Cubarsí	98.73	98.73	94.38	87.34	95.00	66.80
1	Juan Jesus	98.89	98.89	92.85	85.10	94.44	63.48
2	Íñigo Martínez	97.53	97.53	92.74	87.03	93.02	64.42
3	A. Tchouaméni	97.43	97.43	93.61	87.16	100.00	66.02
4	Bremer	97.40	97.40	94.43	89.31	100.00	65.14
5	A. Romagnoli	98.25	98.25	93.79	87.73	100.00	65.04
6	A. Rüdiger	97.27	97.27	92.72	83.84	98.75	61.64
7	J. Giménez	97.07	97.07	92.47	84.52	95.45	63.96
8	Rafael Tolói	93.88	93.88	91.07	89.71	94.12	65.52
9	E. Boyomo	95.65	95.65	89.03	81.48	97.01	62.13
10	Natan	97.43	97.43	89.62	76.92	92.50	63.07

In [55]:
```
fig, tabla = plot_player_similarity(df_players_2425, "J. Bellingham", important_features, n_neighbors=10)
fig.show()

fig.write_image(
    image_path / "similar_players_j_bellingham.png",
    width=1280,
    height=720,
    scale=2
)

display(tabla)
```



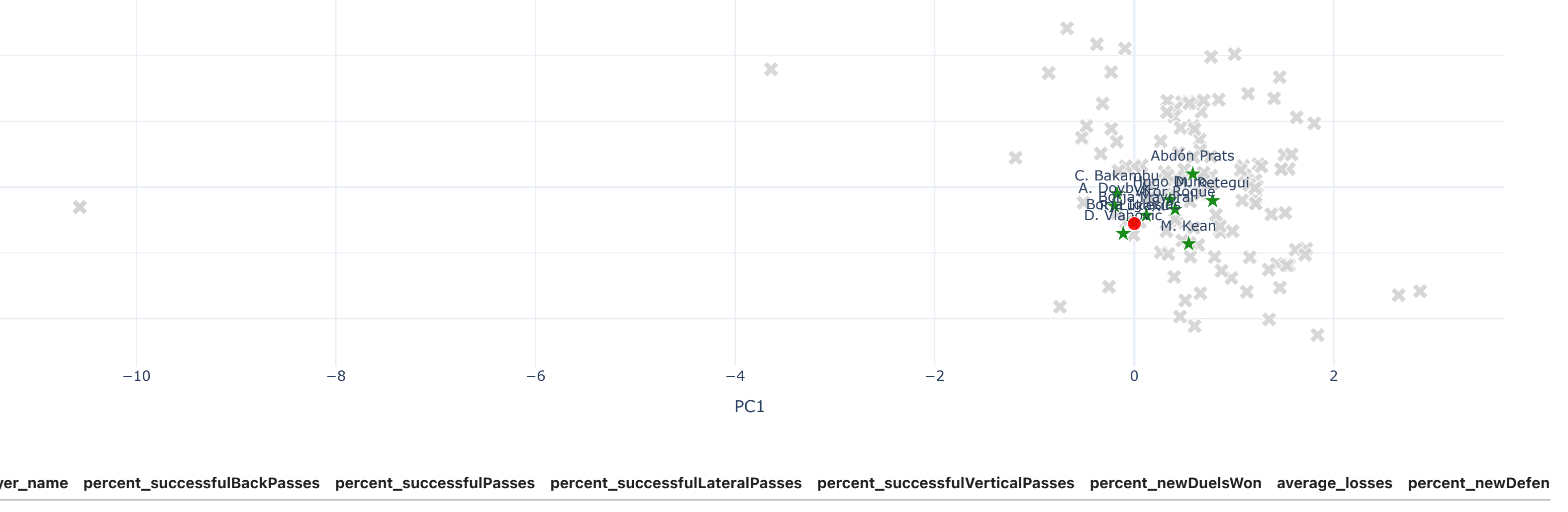
	player_name	total_attackingActions	total_successfulAttackingActions	total_touchInBox	total_successfulDribbles	total_dribbles	total_offensiveDuels	total_shotsOnTarget	total_offensiveDuelsWon	t
0	J. Bellingham	131	59	58	49	60	173	15	104	
1	P. Dybala	141	66	57	53	65	194	17	111	
2	Oihan Sancet	134	66	49	47	66	195	18	94	
3	Álex Berenguer	165	74	46	52	71	185	18	89	
4	D. Man	167	74	60	53	84	160	16	79	
5	S. McTominay	118	52	58	38	59	227	18	104	
6	C. Pulišić	165	76	66	61	77	148	16	91	
7	Aimar Oroz	129	50	46	51	66	246	11	124	
8	A. Colpani	112	48	42	42	59	144	12	66	
9	R. Orsolini	158	75	49	64	81	156	18	86	
10	J. Odgaard	102	44	43	32	43	187	15	80	

In [56]:
```
fig, tabla = plot_player_similarity(df_players_2425, "T. Kubo", important_features, n_neighbors=10)
fig.show()

fig.write_image(
    image_path / "similar_players_t_kubo.png",
    width=1280,
    height=720,
    scale=2
)

display(tabla)
```



	player_name	percent_newDefensiveDuelsWon	percent_newDuelsWon	percent_successfulBackPasses	average_losses	average_interceptions	average_successfulDefensiveAction	average_newDuelsWo
0	T. Kubo	63.29	46.53	90.98	12.32	2.80	5.70	11.1
1	S. Pierotti	52.94	39.71	91.78	11.64	2.58	6.86	10.1
2	F. Miretti	61.68	43.50	95.33	10.46	2.63	7.25	10.5
3	Álex Sola	55.42	38.44	92.45	11.62	2.32	6.09	10.6
4	Samuel Lino	57.14	49.81	97.62	11.20	2.78	7.14	9.9
5	Dani Raba	57.14	42.74	89.23	12.93	3.29	7.10	11.4
6	Adri Embarba	55.56	39.48	89.36	13.31	2.87	6.92	7.9
7	Jofre Carreras	57.73	48.41	89.33	9.99	2.31	5.32	8.1
8	Luis Rioja	57.00	48.99	90.37	11.14	3.37	5.83	7.6
9	G. Simeone	59.38	44.58	89.71	11.25	3.35	6.85	7.9
10	N. Zortea	60.53	45.14	91.59	9.66	2.37	5.63	7.9

In [57]:
```
fig, tabla = plot_player_similarity(df_players_2425, "R. Lukaku", important_features, n_neighbors=10)
fig.show()

fig.write_image(
    image_path / "similar_players_r_lukaku.png",
    width=1280,
    height=720,
    scale=2
)

display(tabla)
```



	player_name	percent_successfulBackPasses	percent_successfulPasses	percent_successfulLateralPasses	percent_successfulVerticalPasses	percent_newDuelsWon	average_losses	percent_newDefen
0	R. Lukaku	90.35	75.54	74.32	74.32	29.37	8.95	
1	D. Vlahović	92.00	74.31	73.74	73.74	31.21	8.67	
2	Borja Iglesias	89.32	75.92	73.75	73.75	29.58	8.44	
3	C. Bakambu	93.75	71.23	68.75	68.75	28.68	9.97	
4	Vitor Roque	92.06	76.09	72.09	72.09	33.43	8.81	
5	M. Kean	83.87	77.61	82.26	82.26	32.90	8.53	
6	Hugo Duro	88.07	77.89	78.95	78.95	33.33	9.16	
7	A. Dovbyk	84.17	78.57	78.95	78.95	27.75	9.42	
8	M. Retegui	93.50	78.93	82.86	82.86	27.61	10.46	
9	Borja Mayoral	100.00	80.36	72.73	72.73	26.77	9.21	
10	Abdón Prats	100.00	76.92	72.73	72.73	29.03	10.40	